

Python

K19G

2026-09-02

Table of contents

Everything Python — Comprehensive Syllabus	14
How to read this book (pick a path)	14
1. Language	14
2. Standard Library	15
3. Projects	15
How chapters are written	15
Prerequisites	16
Conventions	16
Formats	16
Acknowledgments	16
 Language	 17
 Intro	 18
 Why Python Exists	 19
 Why Python Exists	 20
Mental model	20
What Python was designed to do	21
1. Read well	21
2. Stretch from a one-liner to a system	21
3. Call out to faster code	21
Who uses it (and why that matters to you)	22
What Python is not	22
Python + CS	22
Exercises	22
Further reading	22
 Language vs CPython	 23
 Language vs CPython vs Ecosystem	 24
Mental model	24
1. The language	24
2. The implementation (CPython)	25
3. The ecosystem	25
One file, three layers	25
Implementations you will meet	26
Exercises	26

Further reading	26
How to Read This Book	27
How to Read This Book	28
The 3-layer practice	28
What a Language chapter expects	28
Paths (short)	28
What is <i>not</i> in Part 01	29
Tooling while you read	29
How this book grows	29
Exercises	30
Further reading	30
2026 Landscape	31
The 2026 Landscape	32
Mental model	32
Language and stdlib you will meet early	32
Runtime: GIL and free-threading	33
Tooling defaults in this book	33
Ecosystem gravity (not in the sidebar yet)	33
What we will not chase	34
Exercises	34
Further reading	34
Getting Started	35
Install Python	36
Install Python	37
Mental model	37
Recommended: uv	37
Worked example: a throwaway lab folder	38
Official installer (if you cannot use uv yet)	38
Verify more than the version	39
Windows notes	39
What not to do	39
What “isolated environment” will mean next	39
Next: set up an editor	40
Exercises	40
Further reading	40
Editors and Fonts	41
Editors and Fonts	42
Mental model	42
What every setup must do	42

A file to open in every editor	43
Fonts (do this once, for every editor)	43
Fonts that work well for Python	43
Install a font (macOS)	44
Install a font (Linux)	44
VS Code (and Cursor, Antigravity, Windsurf, VSCodium)	45
Install the app	45
Extensions (keep this short)	45
Point the editor at the project interpreter	46
User <code>settings.json</code>	46
Where each fork keeps settings	47
Integrated terminal	47
Run and debug	47
Zed	47
Sublime Text	48
Other popular desktop editors	49
Shared habits (any editor)	50
Pitfalls	50
Exercises	51
Further reading	51
CLI-Only Setup	52
CLI-Only Setup	53
Mental model	53
0. Interpreter (same as Install)	53
1. Fonts in a terminal (when you <i>do</i> have a window)	54
2. Choose an editor (pick one)	54
nano (lowest friction)	55
micro	55
Helix (recommended CLI “IDE”)	55
Neovim (if you already speak Vim)	57
Emacs (no X)	57
3. tmux (SSH that survives a drop)	58
4. Run, lint, test — no GUI required	58
5. Remote folders vs remote terminals	58
Complete check (copy this)	59
Pitfalls	59
Exercises	59
Further reading	59
REPL and Scripts	60
REPL and Scripts	61
Mental model	61
Start the REPL	61
Explore with <code>help</code> and <code>dir</code>	62
Worked example: ask a string what it can do	62

Run a file	63
<code>python -c</code> and <code>uv run</code>	63
Scripts vs modules (preview)	63
Which surface when	64
Pitfalls	64
Exercises	64
Further reading	64
First Program	65
First Program	66
Mental model	66
Type this	66
What each line is doing	67
Worked example: change the input, predict the output	67
Read from the command line	67
Worked example: the mistakes you want to see	68
A mistake worth making	68
Add the other direction	69
Python + maths	69
Pitfalls	69
Exercises	69
Further reading	70
Project Layout	71
Project Layout	72
Mental model	72
Start with <code>uv</code>	72
What belongs in <code>pyproject.toml</code>	73
Names and files	73
One environment per project	73
Worked example: a function plus a test	74
When to add tools	74
Pitfalls	74
Exercises	75
Further reading	75
Syntax and Types	76
Names and Assignment	77
Names and Assignment	78
Mental model	78
Minimal example	78
Assignment is not math	79
Several names, one statement	79
Worked example: swap without a temporary	80

What is a legal name	80
Mutation vs rebinding (preview)	81
Worked example: two empty lists	81
Using a name too early	82
Python + CS	82
Pitfalls	82
Exercises	82
Further reading	82
Numbers and Bool	83
Numbers, Bool, and None	84
Mental model	84
Integers	84
Worked example: leftover pizza	85
Floats	85
Bool	86
None	87
Operators and types	87
Python + maths	88
Pitfalls	88
Exercises	88
Further reading	88
Strings	89
Strings	90
Mental model	90
Minimal example	90
Quotes and escapes	91
Slices	91
Useful methods	92
Worked example: messy input	92
Worked example: initials	93
Unicode is not bytes	93
Immutability	94
Python + CS	94
Pitfalls	94
Exercises	95
Further reading	95
Control Flow	96
if and match	97
if and match	98
Mental model	98

if	98
Worked example: letter grades	99
Combining conditions	100
Ternary expression	100
match (3.10+)	100
Worked example: match on types and literals	101
When to stay with if	102
Python + CS	102
Pitfalls	102
Exercises	102
Further reading	103
Loops	104
Loops	105
Mental model	105
for over a string or a list	105
for and range	106
Worked example: sum 1 to 100	107
while	107
Worked example: first 10 Fibonacci numbers	107
break, continue, else	108
Looping with an index	109
Nested loops	109
Python + CS	110
Pitfalls	110
Exercises	110
Further reading	110
Comprehensions	111
Comprehensions	112
Mental model	112
Lists	112
Worked example: squares of odd numbers	113
Sets and dicts	113
Worked example: invert a dict	113
Nested	114
Generators (preview)	114
When not to	114
Python + CS	115
Pitfalls	115
Worked example: the <code>[[]] * 3</code> trap	115
Exercises	115
Further reading	115

Functions	116
def and Arguments	117
def and Arguments	118
Mental model	118
Minimal example	118
Kinds of arguments	119
Worked example: defaults are bound once	120
* and / (preview)	120
Arbitrary arguments	121
Return	121
Python + CS	122
Pitfalls	122
Exercises	123
Further reading	123
Scope	124
Scope	125
Mental model	125
Minimal example	125
Assignment makes a name local	126
Worked example: UnboundLocalError	127
Built-ins	128
Closures (preview)	128
Python + CS	129
Pitfalls	129
Exercises	129
Further reading	130
First-Class Functions	131
First-Class Functions	132
Mental model	132
Pass a function	132
Worked example: sort by the number after =	133
Return a function	134
lambda	134
Small higher-order tools	135
Python + CS	135
Pitfalls	135
Worked example: the late-binding loop	135
Exercises	136
Further reading	136

Data Structures	137
Lists and Tuples	138
Lists and Tuples	139
Mental model	139
Lists	139
Worked example: stack with a list	140
Tuples	141
Shallow copies	142
Worked example: a 3×3 grid of independent rows	142
Python + CS	142
Pitfalls	143
Exercises	143
Further reading	143
Dicts and Sets	144
Dicts and Sets	145
Mental model	145
Dicts	145
Worked example: count letters	146
Sets	146
Worked example: names in both lists	147
What can be a key	147
dict vs list vs set — pick	148
Python + CS / maths	148
Pitfalls	148
Exercises	149
Further reading	149
Unpacking	150
Unpacking	151
Mental model	151
Sequences	151
Worked example: first and last name	152
Swapping and returning	152
Stars in calls	153
Merging (3.9+)	153
Nested	154
Pitfalls	154
Exercises	155
Further reading	155
Modules and Packages	156
Intro	157

Modules and Packages	158
Planned chapters	158
See also	158
 Objects and Classes	 159
Intro	160
Objects and Classes	161
Planned chapters	161
 Errors and I/O	 162
Intro	163
Errors and I/O	164
Planned chapters	164
See also	164
 Stdlib Essentials	 165
Intro	166
Stdlib Essentials	167
Planned chapters	167
 Typing	 168
Intro	169
Typing	170
Planned chapters	170
 Testing	 171
Intro	172
Testing	173
Planned chapters	173
 Idioms	 174
Intro	175

Idioms	176
Planned chapters	176
 Concurrency	 177
Intro	178
Concurrency	179
Planned chapters	179
 Internals	 180
Intro	181
Internals	182
Mental model	182
Chapters	182
How to work these pages	183
What this is not	183
Further reading	183
 Object Model	 184
Object Model	185
Mental model	185
Minimal example	185
Type objects are objects	185
Identity vs equality	186
Reference counts (default build)	187
Everything has a type, including functions	187
CS connection	187
Pitfalls	187
Exercises	188
Further reading	188
 Names, Namespaces, Frames	 189
Names, Namespaces, and Frames	190
Mental model	190
Minimal example	190
LEGB at runtime	191
Closures are cells	191
Modules are dicts	191
Stack traces are frame chains	192
CS connection	192
Pitfalls	192
Exercises	192
Further reading	193

Bytecode and Eval	194
Bytecode and Eval	195
Mental model	195
Minimal example	195
compile and eval	196
Why this matters	196
Specialization (3.11+)	197
CS connection	197
Pitfalls	197
Exercises	197
Further reading	198
GIL and Free-Threading	199
GIL and Free-Threading	200
Mental model	200
What the GIL does and does not do	200
See it	201
3.13–3.14 timeline	201
Multiple interpreters (PEP 734)	202
When to care	202
CS connection	202
Pitfalls	202
Exercises	202
Further reading	203
Memory and GC	204
Memory and GC	205
Mental model	205
What to use today	205
Planned depth (later)	205
Further reading	206
Import System	207
Import System	208
Mental model	208
What to use today	208
Planned depth (later)	208
Further reading	209
Descriptors and Slots	210
Descriptors and Slots	211
Mental model	211
What to use today	211
Planned depth (later)	212

Further reading	212
Standard Library	213
Intro	214
Standard Library Tour	215
Planned chapters	215
Projects	216
Intro	217
Projects	218
Planned labs	218

Everything Python — Comprehensive Syllabus

A **deep library**, not a single linear course. Use a path below so the sidebar does not feel endless.

Baseline: Python **3.14** · toolchain **uv** + a virtual environment · type hints after the first syntax pass.

This book is **self-contained**. You do not need any other title in this library (Linux, Maths, Networking, Go, ...) to read it. The published sidebar is three parts for now: **Language**, **Standard Library**, and **Projects**. Other domains (CS, maths, data/AI, ops, apps) are parked and will be organized later — they will stay **Python + that domain**, a small experiment *here*, not a second textbook from elsewhere.

How to read this book (pick a path)

Path A - Productive Python (2-4 weeks)

Language: Intro → Getting started → Syntax → Control → Functions
→ Data structures → Modules
→ one small project in Projects

Path B - Fluent Python / internals

Language: Objects → Errors/I/O → Typing → Idioms → Concurrency
→ Internals (object model, bytecode, GIL / free-threading)
→ Standard Library tour

Path C - depth on one topic

Pick one Language section, or the Standard Library map, and work it end-to-end

1. Language

Linux-style tree: **section** → **chapter**, with **Internals** nested under Language.

- **Intro:** why Python exists; language vs CPython vs the ecosystem; how to read this library; the 2026 map (3.14, free-threading, uv).
- **Getting started:** install; editors and fonts (VS Code family, Zed, Sublime, ...); Linux CLI-only; REPL vs file; first program; project layout.
- **Core language:** types, control flow (**match**), functions, lists/dicts/sets, modules and uv, classes, exceptions and **pathlib**.
- **Craft:** stdlib essentials, typing, pytest, idioms, concurrency.

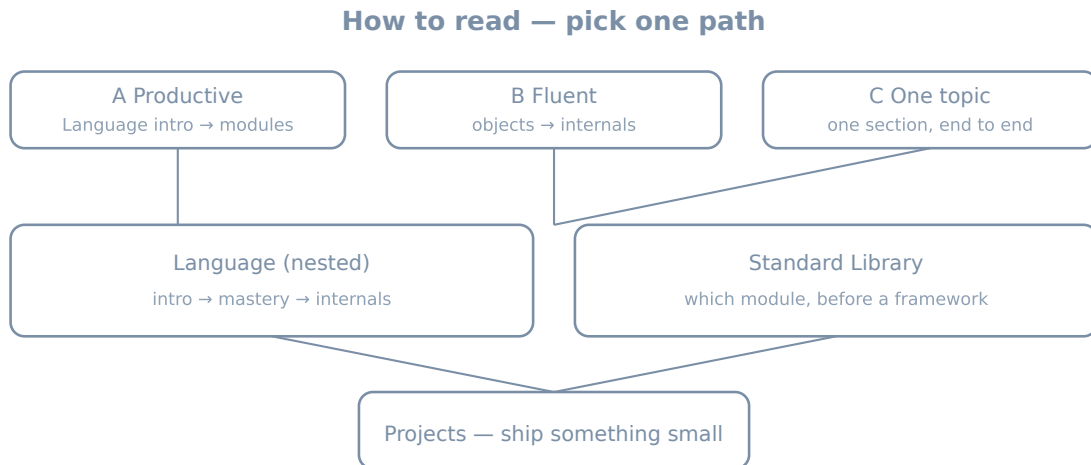


Figure 1: Learning paths

- **Internals:** object model, names and frames, bytecode and eval, GIL and official free-threading (PEP 703 / 779), GC, import system, descriptors.

Start at **Language** → **Intro**. Do not open Internals until you can write a small module and a test.

2. Standard Library

A reference tour: when to reach for which module before a framework. Language → Stdlib essentials is the working set; this part is the map (pathlib, json, datetime, argparse, collections, subprocess, ...).

3. Projects

Build practice that uses Language + the standard library: a temperature CLI, file/rename scripts, word counts, then grow from there.

How chapters are written

Most Language pages include:

1. A one-sentence goal
2. A mental model (ASCII, sometimes SVG)
3. A **complete file** you can save (`# name.py`) and run with `uv run python name.py`

4. Named worked examples, each also a complete file, with expected output
5. Pitfalls
6. An optional **Python + CS** or **Python + maths** note (a small experiment in this book)
7. Exercises
8. Further reading (docs, PEPs, named books — original prose only)

Prerequisites

- A computer, a terminal (the text window where you type commands), and a text editor you already like
- Willingness to type programs, break them, and read the traceback
- No prior Python, and no other book from this library. Some programming experience helps but is not required for Path A

Conventions

- Examples assume **Python 3.14** unless a page says otherwise.
- Prefer **uv** for installing Python and creating environments; **pip + venv** are still shown so you can read older projects.
- Format and lint with **Ruff** when a project exists; do not bikeshed style in the first week.
- Worked examples over API dumps.

Formats

HTML, PDF, and EPUB when built via the monorepo portal.

Acknowledgments

The syllabus is distilled from the official tutorial and from widely used books and courses (Sweigart, Matthes, Downey, Lutz, Slatkin, Ramalho, Beazley, Shaw, and others listed in `content/_planning/syllabus.md`). Chapters are original teaching text for this library, not extracts.

Language

Intro

Why Python Exists

Why Python Exists

After this page you will know what Python is for, who uses it, and what it is *not* — so the rest of Path A has a reason.

Python exists so that people can write **clear programs quickly** on a wide range of machines, then grow those programs without throwing the language away.

You do not need another book to start. A terminal, an editor, and the next section (install) are enough.

Mental model

```
human idea
  |
  v
readable source    ← this is the product Python optimizes
  |
  v
CPython (or another implementation)
  |
  v
your OS, files, network, GPU, browser, robot, ...
```

Guido van Rossum started Python in the late 1980s at CWI in the Netherlands. He wanted a successor to ABC that was practical for real work: extensible in C, good at text and glue, and pleasant to read. The name is from Monty Python, not the snake.

The language succeeded because it sat in a gap:

Pressure	What Python offered
Scripts were too small	Modules, packages, a huge standard library
C / C++ were too slow to write	A high-level language that still calls C
Java / C# needed a heavy runtime story	A single file and a REPL
Shell was great at glue, weak at data	One language for glue <i>and</i> data structures
Research needed a notebook	The same language in a REPL, a script, and a service

What Python was designed to do

1. Read well

Python uses indentation instead of braces. That is not a gimmick: it forces a single visual structure so a file looks like the block tree the interpreter uses.

```
# greet.py
def greet(name):
    if name:
        return f"hello, {name}"
    return "hello"

if __name__ == "__main__":
    print(greet("Ada"))
    print(greet(""))
```

```
uv run python greet.py
```

Expected:

```
hello, Ada
hello
```

There is one obvious layout: the indent *is* the block. Tools such as **Ruff** (a formatter and linter you will meet in Project layout) lock the rest of the style down so reviews talk about behavior, not spaces.

2. Stretch from a one-liner to a system

The same language runs:

- a 5-line rename script
- a pandas notebook
- a FastAPI service
- a Nornir run across a lab
- a training loop on a GPU

You will change *libraries* and *architecture*. You rarely need a second language for the next job.

3. Call out to faster code

Python is often the **control plane**. NumPy, PyTorch, database drivers, and most OS bindings are C, C++, Rust, or Fortran underneath. That is why a slow-looking language dominates scientific computing and AI: the hot loops are not in Python bytecode.

Who uses it (and why that matters to you)

Python is the default glue in:

- **Science and data** — NumPy, pandas, Jupyter
- **Machine learning and AI** — sklearn, PyTorch, JAX, LLM clients
- **Web APIs** — FastAPI, Django
- **Automation** — Ansible (Python under the hood), cloud SDKs, Netmiko
- **Education** — CS50P, MIT 6.100L, almost every intro CS course

If you learn the language *and* how CPython runs it, those domains share one mental model.

What Python is not

- **Not the fastest CPU language.** For a tight inner loop, measure, then move work to NumPy, a native extension, or another language.
- **Not a single-binary culture.** You ship an environment (uv lockfile or a container), not one static file.
- **Not “only scripting.”** That reputation is twenty years out of date. Production services, models, and operators all run Python.
- **Not one implementation.** CPython is what this book means by “Python” unless a page says PyPy, MicroPython, or IronPython.

Python + CS

A language is a **notation plus a machine**. The notation is what you type (`def`, `if`, a list). The machine is the program that carries it out (**CPython** — the next page). Part 01 teaches the notation first. **Internals** teaches the machine, after you can write a small script.

Exercises

1. Write three tasks you personally want Python to do (one script, one data task, one service or automation). Keep the list; Path A should hit at least one.
2. Name one job you would *not* start in Python, and why (performance, binary distribution, browser, kernel, ...).
3. Read the first page of the [official tutorial](#) and list two claims that match this chapter and one that surprised you.

Further reading

- [The Python Tutorial — Whetting Your Appetite](#)
- History notes in the [Python docs](#)

Language vs CPython

Language vs CPython vs Ecosystem

After this page you can hear someone say “Python is slow” and ask: do they mean the language, CPython, or a library?

“Python” is three different things. Mixing them up is the usual reason people argue past each other. You do not need to memorize implementation names today — just keep the three boxes apart.

Mental model

Ecosystem PyPI, conda, wheels, tools

Implementation CPython, PyPy

Language syntax, types,
data model, stdlib spec

1. The language

The **language** is the rules: what a name is, how `for` works, what a `list` guarantees, how `import` is supposed to find a module, which modules the standard library documents.

Those rules live in:

- the [Language Reference](#)
- the [Standard Library](#) docs
- PEPs (Python Enhancement Proposals) when the rules change

If two implementations both run your file and agree on the docs, they are both “Python.”

2. The implementation (CPython)

CPython is the program that actually runs your file. It is written in C. It reads your source, compiles it to **bytecode** (simple instructions for a virtual machine), and runs those instructions. When you type `python` or `uv run python`, you almost always start CPython. This book’s examples assume that, on any OS.

CPython decisions that are *not* the language:

CPython fact	Language fact
There is a GIL in the default build	The language has threads
Free-threaded 3.14 builds can drop the GIL	Same language, different runtime
<code>id(x)</code> is the object’s address	<code>id</code> is unique for the object’s lifetime
Reference counting + a cyclic GC	Objects are garbage-collected

This book’s **Internals** section is about CPython. Other implementations (PyPy, MicroPython) reuse the language and a large chunk of the `stdlib`, then change the machine.

3. The ecosystem

The **ecosystem** is everything you `uv add` or `pip install`: NumPy, FastAPI, pandas, Netmiko, pytest. None of that is “Python the language.” It is software written *in* or *for* Python.

The ecosystem also includes **tools**:

Tool	Job
<code>uv</code>	Install Python, create venvs, lock dependencies (default in this book)
<code>pip + venv</code>	Older, still everywhere
Ruff	Lint and format
<code>ty</code> / Pyright / mypy	Type check
pytest	Test

When a page says “install FastAPI,” that is ecosystem. When it says “`def` creates a function object,” that is language. When it says “bytecode lives in `__code__`,” that is CPython.

One file, three layers

```
# three_layers.py
# language: def, annotation syntax, f-string
def area(radius: float) -> float:
    return 3.14159 * radius * radius
```

```
# ecosystem (only if you installed it):
# import numpy as np

# CPython: this is a function object with a code object
if __name__ == "__main__":
    print(area(2))
    print(area.__code__.co_argcount)

uv run python three_layers.py
```

Expected:

```
12.56636
1
```

You do not need to read `__code__` to use Python. You do need to know it exists so Internals has a place to stand.

Implementations you will meet

Name	Where it shows up
CPython	Default everywhere this book cares about
PyPy	Long-running JIT workloads
MicroPython / CircuitPython	Microcontrollers
Free-threaded CPython	Official in 3.14 (opt-in build); same language, no GIL

Exercises

1. Classify each item as language, CPython, or ecosystem: `list.append`, `numpy.ndarray`, the GIL, `match/case`, `Ruff`, `importlib`.
2. Run `python -c "import sys; print(sys.implementation)"` and read the fields.
3. Explain to a rubber duck why “Python is slow” is a statement about an *implementation* (and usually about *your* code) rather than about `for` loops as a notation.

Further reading

- [Language Reference](#)
- [sys.implementation](#)
- Internals → [Object model](#) (come back after Getting started)

How to Read This Book

How to Read This Book

This is a library, not a single sitting. If you try to read every Language leaf in one go, you will stall. Pick a **path** on the landing page and treat the sidebar as an index, not a to-do list.

The published book is three parts: **Language**, **Standard Library**, **Projects**.

You do not need any other book in this library. When a page uses a terminal, a file path, or a bit of arithmetic, it explains the minimum right here.

The 3-layer practice

```
Read      a short chapter (this book)
|
v
Write     type the examples; change one thing; break it
|
v
Ship      a tiny script, test, or note you would keep
```

Reading without trying the code is tourism. Shipping without reading is copy-paste. Alternate.

A **REPL** is the window where you type Python and see an answer immediately (next section). A **script** is a `.py` file you save and run. Path A uses both.

What a Language chapter expects

1. **Type** the minimal example as a file (first line is `# name.py`). Run `uv run python name.py`. Do not only look at it.
2. Change one name, one operator, or one argument. Predict the result. Run it again.
3. When it fails, read the traceback from the **bottom**. The last line is the error type and message; the lines above are “how we got here.”
4. Do the exercises before opening the next file.
5. If a page has a **Python + maths** or **Python + CS** note, treat it as a small experiment you can try *here*. It is not homework from another book.

Paths (short)

Path	First folders	Stop when
A Productive	Intro → Getting started → Syntax → Control → Functions → Data → Modules	You can write a script that reads a file and prints a summary
B Fluent / internals	Objects → Typing → Idioms → Concurrency → Internals → Standard Library	You can explain a bytecode listing and what the GIL does in 3.14
C One topic	One Language section, or the Standard Library map	You finished that slice

Start with **Path A** unless you already write Python every week. You can switch paths later. You should not run two full paths in parallel the first month.

What is *not* in Part 01

- Frameworks (FastAPI, Django, pandas) — later, if we add those parts
- A Linux course, a maths course, or a networking course

Part 01 is the language, the default toolchain, and enough of how CPython runs to debug your own programs.

Tooling while you read

You have	Use
Nothing yet	Getting started installs <code>uv</code> and a 3.14 interpreter
A desktop	Editors and fonts — VS Code family, Zed, Sublime, ...
SSH / no GUI	CLI-only setup — Helix, Neovim, tmux
A question about a builtin	<code>help(str.split)</code> in the REPL, then the stdlib docs

Pick **one** editor. Do not install a dozen extensions before the first `print`.

How this book grows

The landing page is the **syllabus** for what this book will become. Some Language sections are still **orientation pages** (a map and a planned chapter table) until they are written. Other domains live in `_planning/` until we decide how to organize them.

Exercises

1. Choose Path A or Path B and write it on a sticky note. Do not change it for two weeks.
2. Open the REPL (next section if you do not have one yet) and run `help(print)`. Quit the help pager with `q`.
3. Open **Projects** and note which first lab you will ship after Path A.

Further reading

- Landing page paths (this book's first page)
- [The Python Tutorial](#) — official companion, not a prerequisite

2026 Landscape

The 2026 Landscape

After this page you will know which 3.14 features Path A uses (`match`, `uv`) and which you can ignore until Internals (free-threading, JIT).

This page is a map of what changed around Python **3.13–3.14** so later chapters can point at it instead of repeating release notes. You can follow Path A without absorbing every row.

Mental model

```
Language 3.14
|
+-- default CPython (GIL on)
+-- free-threaded CPython (GIL off, official in 3.14)
+-- optional JIT / faster eval loop (implementation detail)
|
Tooling
+-- uv (install + venv + lock)
+-- Ruff (lint/format)
+-- ty / Pyright / mypy (types)
+-- pytest
```

You can ignore free-threading and the JIT until **Internals** and concurrency. You should **not** ignore `uv` and 3.14 language features.

Language and stdlib you will meet early

Item	Why it matters here
<code>match / case</code>	Pattern matching — taught in Control flow
Deferred evaluation of annotations (PEP 649 / 749)	Typing chapter; <code>from __future__ import annotations</code> is less necessary
<code>pathlib</code> as the default path API	Errors and I/O (file paths as objects, not string glue)
Multiple interpreters (PEP 734)	Internals / concurrency — isolation without a new process

Runtime: GIL and free-threading

For thirty years the **Global Interpreter Lock** meant one thread ran Python bytecode at a time in a process. I/O and C extensions could still overlap; CPU-bound Python threads could not.

Python 3.13 shipped an experimental free-threaded build. **Python 3.14** made that build **official** (PEP 779): same language, optional binary without a GIL.

```
default 3.14      : threads + GIL      (what `uv python install 3.14` gives you)
free-threaded    : threads, no GIL   (opt in; some wheels may lag)
```

This book's default examples use the **GIL build**. Internals explains both. Do not switch a production service to free-threading until you have tests and a reason.

Tooling defaults in this book

Job	Default	Still documented
Get an interpreter	<code>uv python install 3.14</code>	OS packages, python.org
Isolated env	<code>uv venv + uv sync / uv add</code>	<code>python -m venv + pip</code>
Lint / format	Ruff	—
Types	ty or Pyright; mypy still common	—
Tests	pytest	<code>unittest</code> in the stdlib

`pip install` on the system Python is how environments rot. Use a venv from the first project.

Ecosystem gravity (not in the sidebar yet)

These are the usual 2026 defaults when we add domain parts later. They are **not** published chapters now.

Domain	2026 default we teach first	Also exists
HTTP APIs	FastAPI + Pydantic v2	Django, Flask, Litestar
Full web app	Django	FastAPI + a frontend
Data frames	pandas	Polars (mentioned, not required)
Classical ML	scikit-learn	—
Deep learning	PyTorch	JAX, TensorFlow/Keras
LLM apps	official HTTP clients + evals	frameworks that will rename themselves

Domain	2026 default we teach first	Also exists
NetOps	Netmiko → NAPALM → Nornir	Ansible for YAML-first shops
Packaging	pyproject.toml + uv lock	setup.py fossils

Libraries churn. **Jobs** (parse a table, fit a model, push a config, serve JSON) do not. Chapters teach the job, then one current library.

What we will not chase

- Every web framework bake-off
- Vendor-specific “agent OS” products
- Conda vs uv religious wars (uv is the default; conda appears where scientific stacks still need it)

Exercises

1. Run `python3 --version` (or `uv python list` after the next section). Write down major.minor.
2. In one sentence, distinguish “Python 3.14 the language” from “free-threaded CPython.”
3. Name one library you have already heard of (NumPy, FastAPI, pytest, ...) and whether it is language, stdlib, or third-party.

Further reading

- [What’s New in Python 3.14](#)
- [PEP 779](#) — free-threaded CPython official
- [PEP 734](#) — multiple interpreters
- Astral docs for [uv](#)

Getting Started

Install Python

Install Python

After this page you will have a **Python 3.14** program on your computer that *you* installed, and you will know how to ask it its version.

Python itself is a program. When you type `python` (or `uv run python`), the operating system starts that program. It reads your code and carries it out. The usual implementation is **CPython** — that is what this book means by “Python” unless a page says otherwise.

Mental model

```
uv (recommended)
  → downloads CPython 3.14
  → later: creates a private environment per project
  → later: installs third-party packages into that environment
```

Alternatives

```
→ python.org installer
→ OS package (often older; fine to check "is Python here?")
```

A **terminal** is the text window where you type commands and press Enter. On macOS it is Terminal (or iTerm). On Windows it is PowerShell or Windows Terminal. On Linux it is your distribution’s terminal. This book’s commands are meant to be typed there.

This book’s default tool is **uv**. One tool installs interpreters, creates isolated environments, and locks dependencies.

Recommended: uv

Install uv with the official installer.

Linux / macOS:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Close the terminal and open a new one (so PATH picks up the new program), then:

```
uv --version
```

You want a line like `uv 0.x.x`. If the command is “not found,” the installer printed a directory to add to `PATH` — the list of folders your terminal searches for programs. Follow that hint, then try `uv --version` again.

Windows (PowerShell):

```
winget install Astral.uv
```

Or see the [uv install docs](#). Open a **new** terminal after installing.

Then get Python 3.14 and check it:

```
uv python install 3.14
uv python list
uv run python -V
```

You want a line that starts with Python 3.14.

Worked example: a throwaway lab folder

Create a folder so later chapters have a place to live. `mkdir` makes a directory; `cd` moves into it.

```
mkdir -p ~/src/py-lab
cd ~/src/py-lab
uv init --python 3.14
uv run python -c "import sys; print(sys.version)"
```

On Windows PowerShell, `~/src/py-lab` is usually `C:\Users\YourName\src\py-lab`. Create that folder in Explorer if you prefer, then `cd` into it.

`uv init` writes a `pyproject.toml` (project metadata) and a `.python-version` file. From this directory, `uv run python` is **your** interpreter — not a random `python` that happens to be on the system.

Expected kind of output:

```
3.14.x (main, ...)
```

Official installer (if you cannot use uv yet)

Use python.org/downloads. On the installer, enable **Add Python to PATH** if you see that box.

On Debian/Ubuntu you can also install the OS package. It may not be 3.14 — check:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
python3 -V
```

If the OS Python is 3.12 or 3.13, you can still follow Path A. Features that need 3.14 will say so.

Verify more than the version

```
uv run python -c "import sys; print(sys.executable); print(sys.implementation.name)"
```

Field	Healthy result
Version	3.14.x (or 3.12+ if you are stuck)
<code>sys.executable</code>	A path <i>you</i> recognize (uv cache or project <code>.venv</code>), not a mystery copy
Implementation	<code>cpython</code>

`sys.executable` is the full path to the Python program that is running. If it surprises you, you are probably not in the environment you think you are.

Windows notes

Prefer `winget install Astral.uv` or the `python.org` installer, then the same `uv python install 3.14` flow.

Windows sometimes has a **Store stub**: typing `python` opens the Microsoft Store instead of running Python. If that happens, use `py -3.14` or always `uv run python` from your project folder.

What not to do

- Do not `sudo pip install` into the OS Python. That mixes your work with the system’s own tools and is hard to undo.
- Do not use Python 2. Anything that says `print "hello"` (no parentheses) is an artifact.
- Do not fight `conda` *and* `uv` in the same directory until you know why you need `conda`.

What “isolated environment” will mean next

A **virtual environment** (`venv`) is a private copy of Python plus the packages *this* project needs. Project layout creates one. Until then, `uv run python` from `~/src/py-lab` is enough.

Next: set up an editor

This page only installs the interpreter. The editor setup is the **next chapter**, not a section below.

- Desktop (macOS / Linux): [Editors and Fonts](#) — VS Code, Cursor, Antigravity, Zed, Sublime, fonts
- No GUI / SSH: [CLI-Only Setup](#) — Helix, nano, Neovim, tmux

You do not need a dozen extensions before the first `print`.

Exercises

1. Run `uv run python -V` (or `python3 -V`) and write the exact version string in a note.
2. Print `sys.executable` and confirm you recognize the path. It is fine if it is the OS Python *only* when you chose that on purpose.
3. If `uv` is missing, install it *or* write down why you are using the OS Python for now.

Further reading

- [uv documentation](#)
- [Using the Python interpreter](#)
- Next: [Editors and Fonts](#) · [CLI-Only Setup](#)

Editors and Fonts

Editors and Fonts

After this page you can pick **one** editor, point it at this book's toolchain (uv + a project .venv + Ruff), and set a font you can read for hours.

This page is for **macOS and Linux** with a graphical desktop. If you work over SSH or without a GUI, skip to [CLI-only setup](#). Windows notes are short; the same VS Code / Cursor settings apply.

You do not need another book. An **editor** is the program where you type .py files. The **interpreter** is still `uv run python` from [Install](#). Mixing those two up is the usual “it runs here but not there” bug.

Mental model

```
you type in an editor
  |
  v
a .py file on disk      (UTF-8, 4 spaces)
  |
  v
uv run python file.py ← the interpreter this book uses

optional extras (nice, not required on day one)
language server → completions, "go to definition", red squiggles
Ruff            → format and lint on save
.venv          → the editor must pick *this* project's Python
```

Any program that can save a .py file is enough for Path A. The rest of this page is so the extras do not fight you.

What every setup must do

Job	Healthy result
Save	UTF-8, Unix line endings (<code>\n</code>), spaces not tabs
Run	A terminal can <code>cd</code> to the project and <code>uv run python hello.py</code>
Interpreter	Completions come from .venv, not /usr/bin/python3
Format	Ruff, when you have a <code>pyproject.toml</code>

Pick **one** editor. Do not install two language servers that both format on save.

A file to open in every editor

Save this as `hello_env.py` in `~/src/py-lab` (create that folder with `uv init` if you have not):

```
# hello_env.py
import sys

print(sys.version.split()[0])
print(sys.executable)

cd ~/src/py-lab
uv run python hello_env.py
```

You want 3.14.x and a path that contains `.venv` or `uv`'s cache — not a mystery system Python. After you configure the editor, run the same file from the editor's Run / terminal and compare the two paths.

Fonts (do this once, for every editor)

Code is easier to read when `0/0`, `1/1/I`, and `=/-` cannot be confused. Use a **monospace** font (every character the same width). Do not use a proportional font (Times, Arial, the system UI font) for Python.

Fonts that work well for Python

Font	Why people pick it	Ligatures	Notes
JetBrains Mono	Default recommendation in this book	optional	Clear <code>0/0</code> , free, everywhere
Cascadia Code / Cascadia Mono	Ships with Windows Terminal; also on macOS/Linux	Code has them; Mono does not	Good if you already have it
IBM Plex Mono	Slightly softer, still unambiguous	no	Nice on high-DPI screens
Iosevka	Narrow — more columns on a laptop	optional	Tweakable if you build it
Commit Mono	Designed for reading diffs	optional	Very even weights
Fira Code	Famous ligatures (<code>!=</code> becomes <code>≠</code>)	yes	Only if you <i>like</i> ligatures

Font	Why people pick it	Ligatures	Notes
Hack / Source Code Pro	Old reliable	no	Fine if already installed

Ligatures merge character pairs (`==`, `->`, `!=`) into one glyph. They do not change the file. This book's listings use the raw characters. If ligatures confuse you, pick a font without them, or turn them off.

Nerd Fonts (JetBrainsMono Nerd Font, ...) add icons for status lines and file trees. Useful in Neovim / some terminals. Not required in VS Code or Zed.

Size: 13–15 pt on a laptop, 14–16 on a large monitor. Add a little line height if the font feels cramped.

Install a font (macOS)

Homebrew (cask names as of 2026):

```
brew install --cask font-jetbrains-mono
# optional icons for CLI editors:
# brew install --cask font-jetbrains-mono-nerd-font
```

Or download from [JetBrains Mono](#), open the `.ttf` files, click Install.

Install a font (Linux)

Debian / Ubuntu:

```
sudo apt update
sudo apt install fonts-jetbrains-mono fonts-firacode
```

Fedora:

```
sudo dnf install jetbrains-mono-fonts fira-code-fonts
```

Arch:

```
sudo pacman -S ttf-jetbrains-mono ttf-fira-code
```

Then log out and back in (or restart the editor) so the font list refreshes. In GNOME you can also drop `.ttf` files into `~/.local/share/fonts/` and run `fc-cache -f`.

VS Code (and Cursor, Antigravity, Windsurf, VSCodium)

Visual Studio Code is the common desktop editor for Python. **Cursor**, **Google Antigravity IDE**, **Windsurf**, and several others are **VS Code-shaped**: same settings keys, same Command Palette (**Ctrl+Shift+P** / **Cmd+Shift+P**), usually the same extension IDs. **VSCodium** is VS Code without Microsoft branding or telemetry; extensions come from Open VSX, not the Microsoft Marketplace.

Antigravity (Google, late 2025+) has more than one surface. Use **Antigravity IDE** (the VS Code-like editor) for this page. The agent-only desktop is not a Python editor.

Install the app

macOS:

```
brew install --cask visual-studio-code
# or: brew install --cask cursor
```

Or download from code.visualstudio.com.

Linux: prefer the vendor **.deb** / **.rpm** / official tarball so **code** lands on **PATH**. Flatpak works; the integrated terminal then needs Flatpak's extra permissions to see **uv** in **~/.local/bin**.

Open the **folder** **~/src/py-lab**, not a single file. File → Open Folder. Language servers attach to a folder.

Extensions (keep this short)

In the Extensions view, install:

Extension	Publisher / id	Job
Python	Microsoft ms-python.python	Interpreter picker, run, tests
Ruff	Astral charliermarsh.ruff	Lint + format

The Python extension pulls **Pylance** (completions / type hints) and **Python Debugger** (debugpy). You do not need Pylint, Flake8, Black, autopep8, or isort — Ruff covers that job.

Cursor / Windsurf / Antigravity IDE: search the same names. If an ID is missing, install the Open VSX build of Ruff and a Pyright/Pylance-compatible language server.

Point the editor at the project interpreter

1. Ctrl+Shift+P / Cmd+Shift+P
2. **Python: Select Interpreter**
3. Choose `./.venv/bin/python` (uv creates `.venv` on `uv sync` / `uv add`)

If `.venv` is missing:

```
cd ~/src/py-lab
uv sync
```

Status bar (bottom right) should show `3.14.x ('.venv': venv)` or similar.

User settings.json

Command Palette → **Preferences: Open User Settings (JSON)**. This is a complete file you can paste. It sets the font, Ruff on save, and Python defaults. It does **not** pin a single machine's `.venv` path (that is per-workspace).

```
{
  "editor.fontFamily": "JetBrains Mono, Menlo, monospace",
  "editor.fontSize": 14,
  "editor.lineHeight": 22,
  "editor.fontLigatures": false,
  "editor.tabSize": 4,
  "editor.insertSpaces": true,
  "editor.rulers": [88],
  "editor.formatOnSave": true,
  "files.eol": "\n",
  "files.insertFinalNewline": true,
  "files.trimTrailingWhitespace": true,
  "python.languageServer": "Pylance",
  "python.analysis.typeCheckingMode": "off",
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.codeActionsOnSave": {
      "source.fixAll.ruff": "explicit",
      "source.organizeImports.ruff": "explicit"
    }
  }
}
```

`typeCheckingMode` is off for Path A. Turn it to `basic` when you reach the Typing section.

Optional **workspace** file `.vscode/settings.json` (commit this if the team agrees):

```
{
  "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python"
}
```

Where each fork keeps settings

App	macOS user settings	Linux user settings
VS Code	~/Library/Application Support/Code/User/settings.json	~/.config/Code/User/settings.json
Cursor	.../Cursor/User/settings.json	~/.config/Cursor/User/settings.json
VSCodium	.../VSCodium/User/settings.json	~/.config/VSCodium/User/settings.json
Antigravity IDE / Windsurf	same pattern under that app's name	same

The JSON keys above are the same. Do not copy a whole `User/` folder between forks — only the settings you understand.

Integrated terminal

Terminal → New Terminal. Check:

```
which uv
uv run python -c "import sys; print(sys.executable)"
```

If `uv` is “not found,” your GUI app was started without the same `PATH` as the terminal. On macOS, launch from a terminal once (`code .` or `cursor .`) or add `~/.local/bin` (`uv`’s default) to the shell file the GUI reads. On Linux, put a symlink in `/usr/local/bin` or fix the desktop file’s environment.

Run and debug

- Run the current file: the `Run` button, or right-click → Run Python File in Terminal. Prefer a terminal tab: `uv run python hello_env.py` so you always hit the project interpreter.
- Breakpoints: click left of a line number, then Run → Start Debugging (F5). The debugger uses `debugpy`; pick the `.venv` interpreter first.

Zed

[Zed](#) is a native editor (not a VS Code fork). Python works without an extension pack. Recent Zed uses **basedpyright** for navigation and **Ruff** for format/lint. It discovers `uv` / `.venv` toolchains itself.

macOS:

```
brew install --cask zed
```

Linux: see zed.dev/download. Then from a project:

```
cd ~/src/py-lab
zed .
```

Open Settings (Cmd+, / Ctrl+,) → open `settings.json`. Complete starter:

```
{
  "buffer_font_family": "JetBrains Mono",
  "buffer_font_size": 14,
  "buffer_font_features": {
    "calt": false
  },
  "tab_size": 4,
  "hard_tabs": false,
  "format_on_save": "on",
  "languages": {
    "Python": {
      "language_servers": ["basedpyright", "ruff"],
      "formatter": {
        "language_server": {
          "name": "ruff"
        }
      },
      "code_actions_on_format": {
        "source.organizeImports.ruff": true
      }
    }
  }
}
```

`calt: false` turns ligatures off. If you *want* Fira Code ligatures, set `"buffer_font_family": "Fira Code"` and `"calt": true`.

If import resolution is wrong, open the toolchain selector and pick `.venv`. Zed activates that `venv` in new integrated terminals.

Optional: later you can swap `basedpyright` for `ty` (Astral) by putting `"ty"` first and `"!basedpyright"` in `language_servers`. Path A does not need that.

Sublime Text

[Sublime Text 4](#) is fast and stays out of the way. Language intelligence is a plugin, not built in.

1. Install Sublime (macOS: `brew install --cask sublime-text`; Linux: vendor repo).
2. Install **Package Control** (Tools → Install Package Control).
3. Command Palette (Cmd+Shift+P / Ctrl+Shift+P) → **Package Control: Install Package**, then:
 - **LSP**
 - **LSP-pyright** (or LSP-basedpyright)
 - **Ruff** (or **LSP-ruff**)
 - **Terminus** (an in-editor terminal)

Preferences → Settings — complete user file:

```
{
  "font_face": "JetBrains Mono",
  "font_size": 14,
  "font_options": ["no_calt"],
  "translate_tabs_to_spaces": true,
  "tab_size": 4,
  "rulers": [88],
  "ensure_newline_at_eof_on_save": true,
  "trim_trailing_white_space_on_save": "all",
  "default_line_ending": "unix"
}
```

Point Pyright at the project venv. Project → Save Project As... then edit the `.sublime-project` file:

```
{
  "folders": [{ "path": "." }],
  "settings": {
    "LSP": {
      "LSP-pyright": {
        "settings": {
          "python.pythonPath": "${folder}/.venv/bin/python"
        }
      }
    }
  }
}
```

Run from Terminus or an external terminal: `uv run python hello_env.py`.

Other popular desktop editors

You only need one. This table is so the names are not mysteries.

Editor	Where it fits	Python setup in one line
PyCharm Community	Full IDE: debugger, refactors, Django later	New Project → existing <code>~/src/py-lab</code> → interpreter <code>.venv</code> (uv). Enable Ruff plugin. Font: Settings → Editor → Font.
PyCharm Professional	Same + web/DB tools	Same interpreter story; skip until you need those tools.
VSCodium	VS Code without Microsoft Marketplace	Same <code>settings.json</code> ; get Python + Ruff from Open VSX .
Cursor	VS Code + AI chat/inline	Same Python + Ruff extensions and settings as VS Code. Still pick <code>.venv</code> .
Windsurf	Another VS Code-shaped AI IDE	Same.
Antigravity IDE	Google's VS Code-shaped IDE	Same settings keys; install Python + Ruff if the catalog has them.
Nova (macOS)	Native Mac editor	Install a Python / LSP extension; set interpreter to <code>.venv/bin/python</code> .
Kate / GNOME Text Editor / Geany (Linux)	Distro defaults	Fine for typing. Add an LSP plugin if you want squiggles; or just use the terminal.
IDLE	Ships with python.org	Fine for a 10-line experiment. Do not stay there.
JupyterLab / VS Code notebooks	Data later	Not Path A. <code>.py</code> files first.

Emacs, Neovim, Helix, and `nano` live on [CLI-only setup](#) — they also work in a GUI terminal.

Shared habits (any editor)

- **One folder = one project.** Open `~/src/py-lab`, not your entire home directory.
- **Format on save** with Ruff once you have a `pyproject.toml`. Until then, four spaces and a newline at the end of the file are enough.
- **Do not** commit `.venv`, `__pycache__`, or editor crash dumps. `.vscode/settings.json` is optional; `.idea/` (PyCharm) is usually gitignored.
- **Two Python**s — the one the editor uses for completions, and the one the terminal runs — must match. When they disagree, print `sys.executable` in both places.

Pitfalls

- Opening a single file instead of a folder: the language server never sees `.venv`.
- Microsoft **Pylint** + Ruff both enabled: double squiggles. Disable Pylint.
- Formatters fighting: Black *and* Ruff on save. Pick Ruff.

- Ligatures that make `==` look like a single character when you are still learning. Turn them off for a month.
- A GUI app that cannot see `uv` because `PATH` was only set in `~/.bashrc` and you launched from the Dock. Launch once from a terminal, or add `~/.local/bin` to a login-shell file (`~/.zprofile` on macOS `zsh`, `~/.profile` on many Linux desktops).

Exercises

1. Install JetBrains Mono (or name the monospace font you already use). Set it in your editor. Type `00 11I == !=` and check you can tell them apart.
2. Open `~/src/py-lab` as a **folder**. Run `hello_env.py` from the editor terminal. Confirm `sys.executable` matches `uv run python hello_env.py`.
3. Enable format-on-save with Ruff. Mess up the spacing in `hello_env.py`, save, and watch it snap back.

Further reading

- [VS Code Python](#)
- [Zed Python](#)
- [Ruff editor integration](#)
- [JetBrains Mono](#)
- Next: [CLI-only setup](#) if you have no GUI, else [REPL and scripts](#)

CLI-Only Setup

CLI-Only Setup

After this page you can edit and run Python on a **Linux or macOS machine that has no desktop** — SSH into a server, a VM, or a laptop you use only in the terminal.

This book is not a Linux course. You need four programs: a **shell** (already there), **uv**, a **terminal editor**, and a way to run `uv run python file.py`. Optional: a **multiplexer** so a disconnect does not kill the session.

If you have a graphical desktop, you can still use everything here inside the editor’s terminal. The [editors and fonts](#) page stays valid.

Mental model

```
your keyboard
  |
  v
terminal app  (Kitty, Ghostty, iTerm, Linux console, SSH)
  |
  v
shell         (bash, zsh, fish) - reads PATH
  |
  +-- uv / python
  +-- editor (nano, micro, Helix, Neovim, Vim, Emacs)
  +-- optional tmux
```

The **terminal app** draws letters. The **shell** runs commands. The **editor** changes files. Mixing “the terminal is broken” with “Python is broken” wastes an afternoon — print `echo $PATH` and `uv run python -V` first.

0. Interpreter (same as Install)

On the remote or local machine:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Log out and back in, or **source** the line the installer printed. Then:

```
uv python install 3.14
mkdir -p ~/src/py-lab && cd ~/src/py-lab
uv init --python 3.14
uv run python -V
```

uv usually lands in `~/.local/bin`. If uv is not found after install:

```
export PATH="$HOME/.local/bin:$PATH"
```

Put that line in `~/.bashrc` (bash) or `~/.zshrc` (zsh) so every new login has it. That is the whole PATH lesson this page needs.

1. Fonts in a terminal (when you *do* have a window)

On a headless server you cannot change the font — use what the SSH client on your laptop uses. Set **JetBrains Mono** (or JetBrainsMono Nerd Font) in:

Terminal	Where
Ghostty	font-family = JetBrains Mono in ~/.config/ghostty/config
Kitty	font_family JetBrains Mono in ~/.config/kitty/kitty.conf
WezTerm	font = wezterm.font("JetBrains Mono") in ~/.wezterm.lua
Alacritty	font.normal.family: JetBrains Mono in ~/.config/alacritty/alacritty.toml
iTerm2 (macOS)	Settings → Profiles → Text
GNOME Console / Terminal	Preferences → Profile → Text
Linux TTY (no GUI at all)	skip; the firmware font is fine for Path A

Ligatures in a terminal are optional. Nerd Fonts matter only if your Neovim theme draws icons.

2. Choose an editor (pick one)

Editor	Install (typical)	Why pick it
nano	already on most Linux	Keys are on the screen. Fine for 30 lines.
micro	see below	nano-like, mouse, syntax highlight
Helix	see below	Modal, LSP built in — best “one binary” CLI IDE

Editor	Install (typical)	Why pick it
Neovim	<code>sudo apt install neovim</code>	Modal, huge ecosystem; you must add LSP yourself
Vim	already there as <code>vi</code> / <code>vim</code>	Same family as Neovim; older defaults
Emacs	<code>sudo apt install emacs-nox</code>	Lisp-configurable; <code>emacs-nox</code> is the no-X build

`emacs-nox` and `vim-nox` are the “no GUI toolkit” packages. Use those on a server.

nano (lowest friction)

```
nano hello_env.py
```

Ctrl-O save, Ctrl-X exit. Tabs: put this in `~/.nanorc`:

```
set tabsize 4
set tabstospaces
set linenumbers
set autoindent
```

Then run `uv run python hello_env.py` in the same directory.

micro

```
# Linux (official tarball pattern - check micro-editor.github.io)
curl https://getmic.ro | bash
sudo mv micro /usr/local/bin/
```

macOS: `brew install micro`. Open `micro hello_env.py`. Ctrl-S save. Optional `~/.config/micro/settings.j`

```
{
  "tabsize": 4,
  "tabstospaces": true,
  "rmtrailingws": true,
  "eofnewline": true
}
```

Helix (recommended CLI “IDE”)

Helix talks to language servers without a plugin manager.

macOS: `brew install helix`

Linux: vendor package or helix-editor.com.

Put the language servers on PATH with uv (once per machine):

```
uv tool install ruff
uv tool install basedpyright
# ensure uv's tool bin is on PATH (usually ~/.local/bin)
uv tool update-shell
```

Complete Helix language file — save as ~/.config/helix/languages.toml:

```
[[language]]
name = "python"
language-servers = ["basedpyright", "ruff"]
auto-format = true
formatter = { command = "ruff", args = ["format", "-"] }

[language-server.ruff]
command = "ruff"
args = ["server"]

[language-server.basedpyright]
command = "basedpyright-langserver"
args = ["--stdio"]
```

Complete Helix user file — ~/.config/helix/config.toml:

```
theme = "onelight"

[editor]
line-number = "relative"
cursorline = true
bufferline = "multiple"

[editor.lsp]
display-inlay-hints = false

[editor.file-picker]
hidden = false

[editor.indent-guides]
render = true
```

Use a dark theme (onedark, base16_default_dark) if your terminal is dark.

```
cd ~/src/py-lab
hx hello_env.py
```

:fmt formats. :w saves. :q quits. Space-k shows the hover (if the LSP started — check :log-open if not).

Helix / basedpyright find `.venv` when you open the **project directory** (`hx .`) rather than a file from `$HOME`.

Neovim (if you already speak Vim)

A full Neovim distribution (LazyVim, NvChad, kickstart.nvim) is optional later. For Path A, Debian/Ubuntu neovim plus one small config is enough.

Save as `~/.config/nvim/init.lua` (Neovim 0.11+ can talk to LSP without a plugin; this file stays small):

```
-- ~/.config/nvim/init.lua
vim.opt.number = true
vim.opt.relativenumber = true
vim.opt.expandtab = true
vim.opt.shiftwidth = 4
vim.opt.tabstop = 4
vim.opt.list = true

-- Format with Ruff on save when the binary exists
vim.api.nvim_create_autocmd("BufWritePre", {
  pattern = "*.py",
  callback = function()
    if vim.fn.executable("ruff") == 1 then
      vim.cmd("silent !ruff format %")
      vim.cmd("edit")
    end
  end,
})

cd ~/src/py-lab
nvim hello_env.py
```

`:w` save, `:q` quit, `i` insert, `Esc` back to normal. If you want go-to-definition, install Helix or add basedpyright to Neovim's LSP later — do not block Path A on a plugin manager.

Emacs (no X)

```
sudo apt install emacs-nox # Debian/Ubuntu
# macOS: brew install emacs
```

emacs hello_env.py. `Ctrl-X Ctrl-S` save, `Ctrl-X Ctrl-C` exit. Spaces:

```
;; ~/.emacs.d/init.el
(setq-default indent-tabs-mode nil)
```

```
(setq-default tab-width 4)
(add-hook 'python-mode-hook (lambda () (setq python-indent-offset 4)))
```

M-x `eglot` (Emacs 29+) starts a language server if `basedpyright-langserver` is on PATH (`uv tool install basedpyright`).

3. tmux (SSH that survives a drop)

If you work over SSH, start **tmux** so a laptop sleep does not kill the editor.

```
sudo apt install tmux      # or: brew install tmux
tmux new -s py
```

Detach: `Ctrl-B` then `d`. Reattach: `tmux attach -t py`. That is enough. Nested tmux-inside-tmux is how people get stuck — one layer.

4. Run, lint, test — no GUI required

From `~/src/py-lab`:

```
uv run python hello_env.py
uv run python -c "import sys; print(sys.executable)"
uv add --dev ruff pytest
uv run ruff format .
uv run ruff check .
uv run pytest
```

There is no “Run button.” The commands *are* the run button. Project layout will add `pytest` properly; the lines above are enough to prove the machine works.

5. Remote folders vs remote terminals

Three common shapes:

Shape	What you do
SSH + editor on the server	<code>ssh host</code> , then Helix/Neovim/nano on the remote disk. This page.
SSH + editor on the laptop	VS Code / Cursor / Zed Remote SSH (or <code>tramp</code> in Emacs) edits remote files in a GUI. Use Editors and fonts locally; <code>uv</code> still lives on the server.
No Python on the laptop	Dev container / Codespaces later. Not Path A.

Do not edit the same file in two places without noticing. One editor, one folder.

Complete check (copy this)

```
# check.sh - run from ~/src/py-lab
set -e
command -v uv
uv run python -c "import sys; print(sys.version); print(sys.executable)"
test -f hello_env.py && uv run python hello_env.py
```

Expected: a 3.14 version line and a path you recognize. If `command -v uv` fails, fix `PATH` before touching the editor.

Pitfalls

- Installing Python with `apt` only and never `uv`: you get an older interpreter and no project venv. Fine for “is Python here?”; not this book’s default.
- Editing as **root** (`sudo nano`). Files become unwritable as you. Use your user.
- Windows line endings (`\r\n`) after copying from a laptop. `uv run python` still runs; Ruff will nag. `ruff format` rewrites them if you configure it; or `sed -i 's/\r$//' file.py`.
- Two venvs: one you created with `python3 -m venv` and one with `uv`. Pick `uv`; delete the extra `.venv` and `uv sync`.
- Helix LSP “does nothing”: `basedpyright-langserver` is not on `PATH`. `command -v basedpyright-langserver` must print a path.

Exercises

1. On the machine you will actually use, run the `check.sh` snippet (or the same commands by hand). Write down `sys.executable`.
2. Open `hello_env.py` in `nano` **or** `Helix`, change the first `print`, save, run it. Change it back.
3. If you SSH: start `tmux`, detach, log out, log in, `tmux attach`. Confirm the shell is still in `~/src/py-lab`.

Further reading

- [uv installation](#)
- [Helix](#)
- [Ruff](#)
- Next: [REPL and scripts](#)

REPL and Scripts

REPL and Scripts

Python has two everyday surfaces: a **REPL** (type a line, see a result) and a **script** (a `.py` file you run). Use both.

REPL stands for *Read, Evaluate, Print, Loop*: Python reads what you typed, runs it, prints the result, and waits again.

Mental model

REPL	script
----	-----
<code>uv run python</code>	<code>uv run python hello.py</code>
one statement at a time	whole file, then exit
<code>_</code> holds the last value	no implicit print
for exploring	for anything you want to keep

Start the REPL

From your lab folder (`~/src/py-lab` if you followed Install):

```
uv run python
```

or, without uv:

```
python3
```

You get a prompt `>>>`. That means Python is waiting. Type an expression and press Enter:

```
>>> 2 + 3
5
>>> "py" + "thon"
'python'
>>> 2 + 3
5
>>> _
5
```

`_` is the last result. It is a REPL convenience. Do not use `_` as a real variable in a file.

A few more lines worth typing:

```
>>> 10 * 3
30
>>> 2 ** 8
256
>>> print("hello")
hello
```

Notice the difference: `2 + 3` shows `5` because the REPL prints the value of an **expression**. `print("hello")` prints `hello` because `print` writes to the screen. `print` itself returns `None`; the REPL usually hides `None`.

Exit with `exit()` or Ctrl-D (Linux / macOS). On Windows, Ctrl-Z then Enter. Interrupt a running line with Ctrl-C.

Explore with `help` and `dir`

```
>>> help(len)
```

A pager opens. Read a bit, then press `q` to quit.

```
>>> dir(str)
>>> help(str.upper)
```

`dir` lists names on an object. `help` explains one of them. This is faster than a search engine for “what can a string do?”

Worked example: ask a string what it can do

```
>>> name = "Ada"
>>> name.upper()
'ADA'
>>> name.lower()
'ada'
>>> len(name)
3
```

`name` still holds `"Ada"`. Methods such as `.upper()` return a **new** string; they do not change the original. Strings will go further.

Run a file

Create a file named `hello.py` in the same folder, in the editor you set up (or nano), with exactly this:

```
# hello.py
print("hello, python")
```

Save it. Back in the terminal (not inside the REPL — exit first if you are still at `>>>`):

```
uv run python hello.py
```

Expected:

```
hello, python
```

The process starts, runs the file from top to bottom, and exits. Nothing stays in memory.

If you see `can't open file 'hello.py'`, you are in the wrong folder. `ls` (Linux / macOS) or `dir` (Windows) lists files. `cd` into the folder that contains `hello.py`.

python -c and uv run

One-off snippets without a file:

```
uv run python -c "print(2 ** 10)"
```

Expected: 1024.

From a project directory, prefer `uv run python ...` so you always hit the **project** interpreter, not a random `python3` on `PATH`.

Scripts vs modules (preview)

A file can be run *or* imported by another file. The usual guard:

```
# hello.py
def main():
    print("hello, python")

if __name__ == "__main__":
    main()
```

When you run the file, `__name__` is the special string `"__main__"`, so `main()` runs. When another file imports this one, the function is defined but `main()` does not run. Modules and packages cover this properly. For now, copy the pattern when a file is “the program.”

Which surface when

Task	Surface
“What does <code>divmod(17, 5)</code> return?”	REPL
Trying a slice or a bit of arithmetic	REPL
Anything longer than a few lines	.py file
Something you will run tomorrow	.py file + a project folder

Pitfalls

- Pasting a whole indented block into the REPL is fiddly. Use a file.
- Running `python hello.py` with a *different* interpreter than `uv run python` is a classic “it works on my machine” bug. Check with `uv run python -c "import sys; print(sys.executable)"`.
- The REPL’s `_` will confuse you if you treat it as a real variable in a script.
- Forgetting to save the file, then running it: you are running yesterday’s text.

Exercises

1. In the REPL, compute the number of seconds in 3 days (`3 * 24 * 60 * 60`) without a calculator.
2. Use `dir(list)` and `help(list.pop)` to learn one method you did not know. Try it on `[1, 2, 3]`.
3. Write `hello.py` so it prints your name, run it, then add the `if __name__ == "__main__"` guard and run it again. The output should be the same.

Further reading

- [Interactive mode](#)
- Next: [First program](#)

First Program

First Program

Goal: a small file that **reads a value**, **does arithmetic**, and **prints a result**. Almost every later program is this loop with more names.

Mental model

```
input → names → computation → output
a number   c, f      × 9 / 5 + 32   print
(typed in
the file,
or argv)
```

You give Python data. You give the data **names**. You compute a new value. You show it.

Type this

Save as `celsius.py` in your lab folder:

```
# celsius.py
def c_to_f(celsius):
    return celsius * 9 / 5 + 32

if __name__ == "__main__":
    c = 20
    f = c_to_f(c)
    print(f"{c} C is {f} F")

uv run python celsius.py
```

Expected:

```
20 C is 68.0 F
```

If you see `SyntaxError`, look at the line number in the message. The usual first-week causes are a missing colon after `def` / `if`, or mixed tabs and spaces.

What each line is doing

Piece	Role
<code>def c_to_f(celsius):</code>	Bind a function — a reusable recipe with a name
<code>celsius</code>	A parameter : a name that will hold whatever the caller passes in
<code>return ...</code>	Send a value back to the caller
<code>c = 20</code>	Bind the name <code>c</code> to the integer 20
<code>f = c_to_f(c)</code>	Call the function; bind <code>f</code> to what came back
<code>f"{c} C is {f} F"</code>	An f-string : text with values dropped in between <code>{...}</code>
<code>if __name__ == "__main__":</code>	Run this block only when this file is the program

Worked example: change the input, predict the output

Change `c = 20` to `c = 0` and run again.

Expected:

```
0 C is 32.0 F
```

Now `c = 100`. Expected: `100 C is 212.0 F`. Those two values are the usual check that the formula is not backwards.

The `.0` is there because `/` in Python 3 always produces a **float** (a number with a fractional part), even when the math comes out even.

Read from the command line

Hard-coding 20 is fine for a first run. A useful program takes input from outside the file.

```
# celsius.py
import sys

def c_to_f(celsius):
    return celsius * 9 / 5 + 32

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("usage: celsius.py <number>", file=sys.stderr)
        raise SystemExit(2)
```

```
c = float(sys.argv[1])
print(f"{c} C is {c_to_f(c)} F")
```

```
uv run python celsius.py 36.6
```

Expected something like:

```
36.6 C is 97.88 F
```

What is going on:

- `import sys` brings in a **standard-library** module — code that ships with Python. `sys.argv` is the list of words from the command line.
- `sys.argv[0]` is the script name (`celsius.py`). `sys.argv[1]` is the first word you typed after that (`36.6`).
- `float(...)` turns text into a number. `"36.6"` the string is not something you can multiply; `36.6` the float is.
- `len(sys.argv) != 2` means “the user did not pass exactly one extra word.” We print a usage line and exit.

Worked example: the mistakes you want to see

No arguments:

```
uv run python celsius.py
```

You should see the usage line, not a traceback (because we checked `len`).

A word that is not a number:

```
uv run python celsius.py potato
```

You get a traceback ending in `ValueError`. That is correct for now. Errors and I/O will catch it politely.

A mistake worth making

Change the formula from `+ 32` to `+ 320` and run the 0 and 100 cases. Both answers will be wrong in the same way. Tests (later) exist so this class of bug cannot hide. For today: **check two known values** (`0 C → 32 F`, `100 C → 212 F`) whenever you touch the formula.

Add the other direction

```
# both_ways.py
def c_to_f(celsius):
    return celsius * 9 / 5 + 32

def f_to_c(fahrenheit):
    return (fahrenheit - 32) * 5 / 9

if __name__ == "__main__":
    print(c_to_f(20), f_to_c(68))
    print(f_to_c(c_to_f(20)))

uv run python both_ways.py
```

Expected:

```
68.0 20.0
20.0
```

There and back should land on 20. Putting both functions in the same file is fine.

Python + maths

The same arithmetic on paper is $F = (9 / 5) * C + 32$. Changing the 20 and watching the output is Python *showing* a formula, not a lecture on algebra. You do not need a maths book for this page.

Pitfalls

- Integer division is **not** an issue here: `/` always returns a float in Python 3. `//` is floor division (`17 // 5` is 3).
- `print` returns `None`. Do not write `return print(x)` and expect a number back.
- Mixing tabs and spaces will raise `TabError`. Use spaces; later, Ruff will enforce it.
- `c_to_f` without parentheses is the function object. `c_to_f(20)` is a **call**. You want the call.

Exercises

1. Add `f_to_c` and print both directions for 20 Celsius (there and back should land near 20).
2. Accept a number from the command line, as above. What happens with no arguments? Two arguments? Read the usage line and any traceback.

3. In the REPL, try `c_to_f` after `from` / a copy-paste of the function. Call it with `-40`. That value is the same in both scales — a handy check.

Further reading

- [First steps towards programming](#)
- Next: [Project layout](#)

Project Layout

Project Layout

A single `hello.py` is fine for a page. Anything you will touch tomorrow needs a **directory**, an **environment**, and a `pyproject.toml`.

An **environment** here means a private Python plus the extra packages *this* project needs. Two projects can then use different package versions without fighting.

Mental model

```
py-lab/
  pyproject.toml    project name, Python version, dependencies
  .python-version   3.14 (uv reads this)
  .venv/           the isolated interpreter (do not commit)
  celsius.py        your code (flat is fine at first)
  tests/           later
  README.md         later
```

This book uses **flat-or-small** layouts: `celsius.py` next to `pyproject.toml`. A `src/` tree can wait until a package actually needs one.

Start with uv

```
mkdir -p ~/src/py-lab
cd ~/src/py-lab
uv init --python 3.14 --name py-lab
uv add --dev pytest
uv run pytest
```

If the folder already exists from Install, skip `mkdir` / `uv init` and just add `pytest`.

What those commands did:

Command	Effect
<code>uv init</code>	Wrote <code>pyproject.toml</code> and <code>.python-version</code>
<code>uv add --dev pytest</code>	Recorded <code>pytest</code> as a dev dependency and installed it into <code>.venv</code>
<code>uv run pytest</code>	Ran <code>pytest</code> <i>with that</i> environment

`uv run` is the habit: it always uses this project's interpreter.

What belongs in `pyproject.toml`

A minimal file looks like this (yours may differ slightly):

```
[project]
name = "py-lab"
version = "0.1.0"
requires-python = ">=3.14"
dependencies = []

[dependency-groups]
dev = ["pytest"]
```

Table	Role
[project]	Name, version, runtime dependencies
requires-python	Reject older interpreters
[dependency-groups]	Test/lint tools (uv)

You do not need extra packaging configuration until you publish a package.

Names and files

Do	Do not
<code>celsius.py</code> , <code>temp_convert.py</code> modules named with <code>snake_case</code>	<code>temp_convert.py</code> (space), <code>celsius.py.py</code> <code>ConvertTemp.py</code> as a first habit
keep <code>.venv</code> out of version control	commit a <code>venv</code> or <code>__pycache__</code>

`__pycache__` is a folder Python creates for compiled bytecode. Leave it alone; do not edit it.

One environment per project

```
~/src/py-lab/.venv      ← this project's packages
~/src/other/.venv       ← a different set
/usr/bin/python3        ← leave the system copy alone
```

If `import pytest` works in one project and fails in another, you are in the wrong environment. Print `sys.executable` and look at the path.

Worked example: a function plus a test

celsius.py:

```
# celsius.py
def c_to_f(celsius):
    return celsius * 9 / 5 + 32
```

test_celsius.py (same folder):

```
# test_celsius.py
from celsius import c_to_f

def test_freezing():
    assert c_to_f(0) == 32

def test_boiling():
    assert c_to_f(100) == 212

uv run pytest
```

Expected: two passed tests. **assert** means “this must be true, or fail the test.” That is enough testing for Path A until the Testing section.

If pytest says it cannot import `celsius`, you are not in the folder that contains both files.

When to add tools

Not on day one. When you have three files:

```
uv add --dev ruff pytest
uv run ruff check .
uv run ruff format .
```

Ruff formats and lints (nags about unused names, bad style). Typing tools (`ty`, `Pyright`, `mypy`) wait until the **Typing** section.

Pitfalls

- Two `pyproject.toml` files nested in each other — `uv` and editors get confused. One project root.
- Editing files that live *outside* the project but running `uv run` *inside* it: you are testing a different tree.
- Copying `.venv` between machines. Recreate it with `uv sync`.

- Installing a package with a bare `pip install` in some other terminal. It will land in a different environment. Use `uv add`.

This page does not teach Git. When you are ready to snapshot the folder, any editor's "initialize repository" or `git init` is enough; commit `pyproject.toml` and your `.py` files, not `.venv`.

Exercises

1. Run `uv init` in an empty directory and list the files it created.
2. Add the `c_to_f` tests above and run `uv run pytest`. Break the formula on purpose and watch a test fail, then fix it.
3. Print `sys.executable` from a script and from `uv run python -c "import sys; print(sys.executable)"`. They should match.

Further reading

- [venv tutorial](#) (the stdlib story; we use uv on top)
- [uv projects](#)

Syntax and Types

Names and Assignment

Names and Assignment

After this page you can look at `x = 20` and draw what Python actually did — and you will know why `b = a` does not copy a list.

In Python a name is a **label on an object**, not a box that holds a value. Assignment sticks a label on something that already exists.

Mental model

```
n = 20          # find (or create) the int 20; attach the name n

n      20

n = n + 1       # compute 21; move the name n

n      21
20      (old int, no name, will be cleaned up)
```

Think of sticky notes, not Tupperware. The number 20 is an object. The name `n` is a sticky note on it. `n = n + 1` does not scratch out the 20 and write 21 inside the same box. It computes a *new* object 21 and moves the sticky note.

This is why `b = a` does not copy a list. Both sticky notes point at the **same** object.

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Minimal example

```
# labels.py
celsius = 20
label = celsius
print(celsius, label)
celsius = 21
print(celsius, label)
```

Expected:

```
20 20
21 20
```

Integers never change. Rebinding `celsius` does not touch `label`. `label` still points at 20.

Type it in the REPL if the picture is fuzzy. Watch the two `prints`.

Assignment is not math

```
# increment.py
x = 3
x = x + 1      # legal: the right-hand side uses the old binding first
print(x)
print(x == 4)
```

Expected:

```
4
True
```

`=` is not equality. Equality is `==`.

```
>>> x = 3
>>> x == 3
True
>>> x == 4
False
```

`=` means “bind this name.” `==` means “are these values equal?” Mixing them up is the first week’s classic `SyntaxError` or a condition that is always true.

Several names, one statement

```
# several_names.py
a, b = 1, 2
print(a, b)
a, b = b, a      # swap - the right-hand side is fully evaluated first
print(a, b)
x = y = 0        # both names on the same 0
print(x, y)
```

Expected:

```
1 2
2 1
0 0
```

Worked example: swap without a temporary

In many languages you write `tmp = a; a = b; b = tmp`. In Python the right-hand side `b`, `a` is built first (a pair of values), then unpacked onto the left.

```
# swap.py
left, right = "tea", "coffee"
left, right = right, left
print(left, right)
```

Expected:

```
coffee tea
```

Unpacking is the same idea: match a structure on the right to names on the left. Data structures will go further.

What is a legal name

```
ok      : celsius, c_to_f, _hidden, X2, name2
not ok  : 2x, celsius-temp, class, def, a name
```

Rules you can remember:

- Start with a letter or `_`, then letters, digits, or `_`.
- Names are case-sensitive: `Temp` and `temp` are different names.
- Some words are **keywords** (`if`, `def`, `class`, `return`, ...). They are already spoken for.

Convention: `snake_case` for functions and variables (`c_to_f`, `user_name`), `CapWords` for classes (`Point`, `HttpError`). Ruff will nag you later; start the habit now.

```
>>> help("keywords")
```

That lists every keyword. Try assigning to `class` and read the error.

Mutation vs rebinding (preview)

```
# two_names.py
nums = [1, 2, 3]
also = nums
also.append(4)
print(nums)
print(also is nums)
```

Expected:

```
[1, 2, 3, 4]
True
```

`append` **mutates** the list (changes the object in place). `also = [0]` would **rebind** the name `also` and leave `nums` alone.

After `nums = [1, 2, 3]` and `also = nums`:

```
nums    [1, 2, 3]
also
```

After `also.append(4)`:

```
nums    [1, 2, 3, 4]
also
```

After `also = [0]`:

```
nums    [1, 2, 3, 4]
also    [0]
```

If a result surprises you, draw the arrows. Lists and tuples will spend more time here.

Worked example: two empty lists

```
# two_lists.py
x = y = []      # one list, two names - usually a mistake
x.append(1)
print("shared", y)

x, y = [], []   # two lists
x.append(1)
print("separate", y)
```

Expected:

```
shared [1]
separate []
```

Using a name too early

```
>>> print(score)
Traceback (most recent call last):
...
NameError: name 'score' is not defined
```

`NameError` means “there is no sticky note with that word.” Either you have not assigned it yet, or you typo’d (`celcius` vs `celsius`).

Python + CS

This is the **environment** in a programming-language sense: a mapping from names to objects. Functions will nest those mappings (scope). You can see the idea without another book: after `n = 20`, the name `n` is in the current environment.

Pitfalls

- Using a name before assignment: `NameError`.
- A typo creates a *new* name (`celcius = 20`) and the old one never updates.
- `x = y = []` — both names, **one** list. For two empty lists use `x, y = [], []`.
- `=` in a condition (`if x = 3`) is a `SyntaxError`. You meant `==`.

Exercises

1. Draw arrows for `a = [1]; b = a; a = [2]`. What is `b`? Run it and check.
2. Swap `left` and `right` in one line without a temporary name.
3. Find three words that cannot be names because they are keywords (`help("keywords")`). Try assigning to one and read the error.

Further reading

- [Naming and binding](#)
- Internals → [Names](#), [namespaces](#), [frames](#) after you have functions

Numbers and Bool

Numbers, Bool, and None

After this page you can pick `int` vs `float`, compare things with `True/False`, and test for “no value” with `None`.

Python’s everyday numbers are `int` (whole numbers, unlimited size) and `float` (numbers with a fractional part; binary approximations). `bool` is `True` or `False`. `None` is the single object that means “no value.”

Mental model

<code>int</code>	<code>20, -3, 10**100</code>	<code>exact</code>
<code>float</code>	<code>20.0, 1e-3</code>	<code>approximate</code>
<code>bool</code>	<code>True, False</code>	<code>also 1 and 0 (do not rely on that)</code>
<code>None</code>	<code>None</code>	<code>only one of these exists</code>

`**` is “to the power of.” `10**100` is a 1 followed by 100 zeros. `1e-3` is scientific notation for 0.001.

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Integers

```
# arithmetic.py
print(2 + 3, 2 * 3, 2 ** 10)
print(17 / 5)
print(17 // 5, 17 % 5)
print(divmod(17, 5))
print(int("42"), int(3.9), int(-3.9))
```

Expected:

```
5 6 1024
3.4
3 2
(3, 2)
42 3 -3
```

Operator	Meaning	17 and 5
+ - *	add, subtract, multiply	
/	divide, always a float	3.4
//	divide, drop the fraction toward $-\infty$	3
%	remainder	2
**	power	17 ** 2 is 289

int does not overflow. `10 ** 100` is fine. The cost is memory and time, not a wrap to a negative number.

Convert with `int("42")` and `int(3.9)` (truncates toward zero, so 3).

Worked example: leftover pizza

17 slices, 5 friends, as even as possible:

```
# pizza.py
slices = 17
friends = 5
each, leftover = divmod(slices, friends)
print(f"each {each}, leftover {leftover}")
print(slices == each * friends + leftover)
```

Expected:

```
each 3, leftover 2
True
```

Each friend gets 3; 2 slices remain. `divmod` returns both answers as a pair.

Floats

```
# floats.py
import math

print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(math.isclose(0.1 + 0.2, 0.3))
print(1e6, 2.5 * 2)
```

Expected:

```
0.30000000000000004
False
True
```

```
1000000.0 5.0
```

That is not a Python bug. Computers store fractional numbers in binary. 0.1 cannot be represented exactly, just as 1/3 cannot be represented exactly in decimal (0.333...). For money, use whole cents (int) or `decimal.Decimal`. For science, compare with a tolerance (`math.isclose`).

The official tutorial's [floating-point chapter](#) is worth a full read when 0.1 + 0.2 first surprises you.

Bool

A **boolean** is a yes-or-no value: `True` or `False`. Comparisons produce them.

```
>>> 3 > 2
True
>>> 3 == 2
False
>>> 3 != 2
True
```

`!=` means “not equal.”

```
# truth.py
print(3 > 2, 3 == 2, 3 != 2)
print(True, False)
print(True + True)
print(bool(0), bool(""), bool([]), bool(None))
print(bool(1), bool("x"), bool([0]), bool("False"))
```

Expected:

```
True False True
True False
2
False False False False
True True True True
```

Use `True/False` in comparisons, not `1/0`, even though they work.

Values that are **falsey** (treated as false in an if): `False`, `0`, `0.0`, `""`, `[]`, `{}`, `set()`, `None`. Everything else is **truthy**, including the string `"False"` and the list `[0]`.

```
>>> bool("False")
True
>>> bool([0])
True
```

The string is not empty; the list is not empty. `if` will treat both as true.

None

```
# none_check.py
x = None
if x is None:
    print("no value yet")
print(x is None, x == None) # both True; use `is`
```

Expected:

```
no value yet
True True
```

Compare to `None` with `is`, not `==`. There is only one `None` object; `is` asks “same object?”

Functions that do not **return** actually return None:

```
# silent.py
def silent(n):
    print(n)

if __name__ == "__main__":
    result = silent(3)
    print(result)
```

Expected:

3
None

`print` displayed 3. The function's *return value* is still `None`.

Operators and types

```
# types.py
print(type(20), type(20.0), type(True), type(None))
print(isinstance(True, int))
print(10 ** 100)
print(-17 == (-17 // 5) * 5 + (-17 % 5))
```

Expected:

[illegible]

True

`type` is for exploration in the REPL. Prefer `isinstance` when you write a check — and prefer *not* writing a check when a simple operation will fail clearly on the wrong type.

Python + maths

- `int` grows as far as memory allows — try `10 ** 100` in the REPL.
- `float` is a finite set of binary fractions; `0.1 + 0.2` is the experiment.
- Check `x == (x // y) * y + (x % y)` for a few integers (try `x, y = 17, 5` and `x, y = -17, 5`). That identity is something you *run*, not something we prove here.

Pitfalls

- Mixing `/` and `//` in the same formula by accident.
- `True == 1` is true; `True is 1` is false. Use `is` only for `None` and known singletons.
- `int("3.0")` raises `ValueError`; use `int(float("3.0"))` if you must.
- `1 / 0` raises `ZeroDivisionError`. Same for `1 // 0`.

Exercises

1. Confirm `divmod(n, d) == (n // d, n % d)` for three pairs, including a negative `n`.
2. Find two floats `a, b` where `a + b == a` but `b != 0`. (`1e16` and `1.0` is a hint.)
3. Write a function `is_blank(x)` that is true for `None` and `""` only. Check that `is_blank("False")` is false.

Further reading

- [Numbers](#)
- [Floating-point issues](#)

Strings

Strings

After this page you can build a message, slice a word, and split a line of text — without trying to change a letter in place.

A string is an **immutable sequence of Unicode characters**. Immutable means you cannot change a letter in the existing object; you build a new string. Unicode means letters, emoji, and other writing systems are all **str**, not a special type.

Mental model

```
s = "python"
```

index	0	1	2	3	4	5
	p	y	t	h	o	n
neg	-6	-5	-4	-3	-2	-1

`s[0]` is "p". `s[-1]` is "n" (last character). `s[1:4]` is "yth" — start inclusive, end **exclusive**. The character at index 4 is not included.

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Minimal example

```
# hello_name.py
name = "Ada"
print(f"hello, {name}")
print(name.upper(), name.lower())
print("Ada" in "Ada Lovelace")
print(len(name))
temp = 20
print(f"{temp} C")
print(f"{temp:.1f} C")
```

Expected:

```
hello, Ada
ADA ada
```

```
True
3
20 C
20.0 C
```

f-strings (`f"..."`) are the default way to build messages. The `{name}` is replaced by the value of `name`. `str.format` and `%` still exist; do not start new code with them. `:.1f` means “one digit after the decimal.”

Quotes and escapes

```
# quotes.py
print("it's fine")
print('she said "ok"')
print("line\nbreak")
print(r"C:\new\to")
print("""triple
quotes for a block""")
```

Expected:

```
it's fine
she said "ok"
line
break
C:\new\to
triple
quotes for a block
```

- Use the other kind of quote when the text contains a quote.
- `\n` is a newline. `\t` is a tab.
- A raw string (`r"..."`) is handy for Windows paths and regular expressions, where backslashes are common.
- Triple quotes keep real line breaks.

Slices

```
# slices.py
s = "python"
print(s[:2], s[2:], s[-2:], s[::-2], s[::-1])
print(s[1:4])
print(s)
print(s[2:100])
```

```
try:
    print(s[100])
except IndexError as err:
    print(type(err).__name__, err)
```

Expected:

```
py thon on pto nohtyp
yth
python
thon
IndexError string index out of range
```

Slice	Meaning	On "python"
s[:2]	from the start up to (not including) 2	"py"
s[2:]	from 2 to the end	"thon"
s[-2:]	last two characters	"on"
s[::2]	every second character	"pto"
s[::-1]	step backwards — reverse	"nohtyp"

A slice of a string is a **new** string. The original does not change. Out-of-range *index* raises `IndexError`. Out-of-range *slice* is quietly clipped.

Useful methods

Method	Role
<code>split, join</code>	Break on whitespace (or a separator); glue a sequence
<code>strip, lstrip, rstrip</code>	Remove edges (spaces, newlines)
<code>replace</code>	Return a copy with substitutions
<code>startswith, endswith</code>	Prefix / suffix tests
<code>isdigit, isalpha</code>	Character class (Unicode-aware)
<code>find, index</code>	Search; <code>index</code> raises if missing, <code>find</code> returns -1
<code>count</code>	How many times a substring appears

Worked example: messy input

```
# messy.py
words = "one two\tthree".split()
print(words)
print(", ".join(words))
```

```
print("  Ada  ".strip())
print("banana".replace("a", "o"))
print("hello.py".endswith(".py"))
```

Expected:

```
['one', 'two', 'three']
one, two, three
Ada
bonono
True
```

`split()` with no argument collapses any whitespace. That is what you want for messy input.

`join` is a **string** method. You call it on the separator, not on the list: `", ".join(words)`, not `words.join(", ")`.

Worked example: initials

```
# initials.py
def initials(name):
    parts = name.split()
    letters = [part[0].upper() for part in parts]
    return "".join(letters)

if __name__ == "__main__":
    print(initials("ada lovelace"))
    print(initials("grace hopper"))
```

Expected:

```
AL
GH
```

Unicode is not bytes

```
# unicode.py
print(len("é"))
print("é".encode("utf-8"))
```

Expected:

```
1
b'\xc3\xa9'
```

Text in your program is **str**. Binary (files, sockets, images) is **bytes**. Decode at the edge (when you read); stay in **str** in the middle. Errors and I/O will open files with an encoding. For Path A, just remember: **len** counts characters, not bytes.

Immutability

```
# immutable.py
s = "py"
try:
    s[0] = "P"
except TypeError as err:
    print(type(err).__name__)
s = "P" + s[1:]
print(s)
```

Expected:

```
TypeError
Py
```

You cannot assign to `s[0]`. You build a new string and move the name.

Python + CS

A string is a **sequence**: it has a length, indexing, and you can loop over it (`for ch in s`). Lists are the mutable cousin. The same ideas — first item, last item, reverse, search — work on both.

Pitfalls

- `str.split("")` is illegal; use a list of characters: `list(s)`.
- `+` in a loop to build a huge string is the slow pattern; collect parts in a list and `join`.
- `"10" > "9"` is **lexicographic** (dictionary order: `"1"` vs `"9"`). Convert to `int` before comparing numbers stored as text.
- `"Ada" == "ada"` is `False`. Use `.lower()` on both sides if you want a case-insensitive check.

Exercises

1. Write `initials.py` so `initials("ada lovelace")` prints `AL`. Hint: `split`, take `[0]` of each word, `upper`, `join`.
2. Write `reverse_words.py`: `"all is well" → "well is all"`.
3. Write `count_an.py` that prints `"banana".count("an")` and `"aaa".count("aa")` (the second is 1 — no overlap).

Further reading

- [Text](#)
- [Unicode HOWTO](#)

Control Flow

if and match

if and match

After this page you can choose which code runs with `if`, and you can pick a *shape* of data with `match`.

Control flow chooses **which** code runs. `if` tests a condition. `match` picks a **shape**.

Every python block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Mental model

<code>if / elif / else</code>	<code>match / case</code>
-----	-----
questions about values	questions about structure
<code>x > 0</code>	point is <code>(x, y)</code>
<code>name == "quit"</code>	command is <code>["load", path]</code>

You can write everything with `if`. `match` is clearer when the interesting part is the *shape* of the data (a list of two words, a number that is 0, 1, or 2, ...).

if

```
# sign.py
def sign(n):
    if n > 0:
        return "positive"
    elif n < 0:
        return "negative"
    else:
        return "zero"

if __name__ == "__main__":
    print(sign(-2), sign(0), sign(3))

uv run python sign.py
```

Expected:

```
negative zero positive
```

- `if` runs its block when the condition is true.
- `elif` is “else if” — tried only when the previous tests failed.
- `else` runs when none of the tests above succeeded.

The condition is anything truthy or falsey (see Numbers and Bool). Prefer explicit comparisons for numbers (`n != 0` rather than `if n`) when 0 is a real value you care about.

Indentation is how Python sees the block. The lines under `if` must be indented the same amount (four spaces is the convention).

```
# even_odd.py
n = 4
if n % 2 == 0:
    print("even")
else:
    print("odd")
```

Expected:

```
even
```

`%` is remainder. Even numbers have remainder 0 when divided by 2.

Worked example: letter grades

```
# letter_grade.py
def letter_grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

if __name__ == "__main__":
    print(letter_grade(95), letter_grade(80), letter_grade(59))

uv run python letter_grade.py
```

Expected:

```
A B F
```

Order matters. If you test `>= 60` first, a 95 would become a D. Test the strictest cut first.

Combining conditions

```
# in_range.py
score = 85
name = "Ada"

if 0 <= score <= 100:
    print("in range")

if name and name.startswith("A"):
    print("starts with A")
```

Expected:

```
in range
starts with A
```

and needs both sides true. or needs one side true. not flips true/false.

Python lets you write `0 <= score <= 100` the way you would on paper. You do not need `score >= 0` and `score <= 100`, though that works too.

Ternary expression

```
# ternary.py
n = -3
label = "ok" if n >= 0 else "negative"
print(label)
```

Expected:

```
negative
```

This is an **expression**: it produces a value. Use it for a single value, not for two statements.

match (3.10+)

match looks at a value and asks “does it look like this?”

```
# describe.py
def describe(command):
    match command:
        case ["quit"]:
            return "bye"
        case ["load", path]:
            return f"loading {path}"
        case ["move", x, y] if isinstance(x, int) and isinstance(y, int):
            return f"to {x},{y}"
        case _:
            return "unknown"

if __name__ == "__main__":
    print(describe(["load", "map.txt"]))
    print(describe(["move", 2, 3]))
    print(describe(["nope"]))
```

Expected:

```
loading map.txt
to 2,3
unknown
```

- `case ["load", path]:` means “a sequence of two items; call the second one `path`.”
- `path` is **bound** (like assignment). After this case matches, you can use `path`.
- `if isinstance(...)` after a pattern is a **guard**: the case only wins if that extra test is true.
- `case _:` is the default. `_` means “I do not need the value.”

Worked example: match on types and literals

```
# kind.py
def kind(value):
    match value:
        case 0 | 1 | 2:
            return "small int"
        case int():
            return "other int"
        case str() as s if s.strip() == "":
            return "blank"
        case str():
            return "text"
        case None:
            return "missing"
        case _:
            return "other"
```

```
if __name__ == "__main__":
    print(kind(1), kind(99), kind("  "), kind("hi"), kind(None))
```

Expected:

```
small int other int blank text missing
```

`int()` here is a **class pattern**, not a call that converts the value. `0 | 1 | 2` is an or-pattern: any of those three.

If you put `case int(): before case 0 | 1 | 2:`, the small integers would never be reached. First match wins.

When to stay with if

- One or two boolean tests (`if user and user.is_active`)
- Ranges (`if 0 <= x < n`)
- Anything that is clearer as a sentence than as a shape

Reach for `match` when you are unpacking a command, a pair, or a few known variants.

Python + CS

`if` is **selection**. `match` is selection plus **destructuring** (pull the pieces out while you test the shape). You can think of it as “look at this value, name its parts, continue.” No other book required.

Pitfalls

- `case other:` binds *everything* and swallows later cases. Put a catch-all last, or use `_` if you do not need the value.
- `case ["load", path]:` does not match `"load"` the string — only a sequence of two items.
- `==` vs `is`: `if x is None` for missing values; `if x == 0` for zero.
- Forgetting the colon after `if / case` is a `SyntaxError`.
- An `if` whose body is not indented is a `SyntaxError` or a surprising “this always runs.”

Exercises

1. Save `letter_grade.py` and add a few more `prints` (0, 60, 100). Then, if you want, rewrite the A/B/C/D/F table as `match on score // 10`.
2. Write `parse.py` with `parse(["add", 1, 2])` and `parse(["neg", 5])` using `match`. Reject anything else with a clear string. Run it.
3. What does `match 3: case int(x): ...` bind `x` to? Try it in the REPL, then as a tiny file.

Further reading

- [if statements](#)
- [match statements](#)

Loops

Loops

After this page you can walk through a string or a list with **for**, count with **range**, and stop a loop with **break**.

A loop repeats work. Python's **for** walks a **sequence** (a string, a list, a **range**, ...). **while** repeats until a condition is false.

Every **python** block below is a complete file. The first line is the filename. Save it and run **uv run python that_file.py**.

Mental model

```
for item in collection:      "for each"
    |
    v
body    → next item → ...

while condition:             "as long as"
    |
    v
body    (must change the condition or break)
```

Reach for **for** first. **while** is for “I do not know how many times” (read until a blank line, wait for a flag, ...).

for over a string or a list

```
# letters.py
for ch in "py":
    print(ch)
```

Expected:

```
p
y
```

Each time around, **ch** is bound to the next character. Then the body runs.

```
# words.py
for word in ["ada", "grace", "alan"]:
    print(word, len(word))
```

Expected:

```
ada 3
grace 5
alan 4
```

for and range

`range` is a sequence of integers. It is not a list until you ask for one.

```
# ranges.py
print("range(3):")
for i in range(3):
    print(i)

print("range(2, 8, 2):")
for i in range(2, 8, 2):
    print(i)

print("as a list:", list(range(3)))
```

Expected:

```
range(3):
0
1
2
range(2, 8, 2):
2
4
6
as a list: [0, 1, 2]
```

Call	Values
<code>range(3)</code>	0, 1, 2 (stop is exclusive)
<code>range(2, 8)</code>	2, 3, 4, 5, 6, 7
<code>range(2, 8, 2)</code>	2, 4, 6 (step 2)

`list(...)` materializes the range when you need to *see* it. In a `for`, you do not need that.

Worked example: sum 1 to 100

```
# sum_to_100.py
total = 0
for n in range(1, 101):
    total = total + n
print(total)
print(sum(range(1, 101)))
```

Expected:

```
5050
5050
```

while

```
# countdown.py
n = 3
while n > 0:
    print(n)
    n -= 1
```

Expected:

```
3
2
1
```

`n -= 1` means `n = n - 1`. If you forget it, `n` stays 3 and the loop never ends. Interrupt with Ctrl-C.

Worked example: first 10 Fibonacci numbers

Each number is the sum of the previous two: 0, 1, 1, 2, 3, 5, ...

```
# fibonacci.py
a, b = 0, 1
count = 0
while count < 10:
    print(a, end=" ")
    a, b = b, a + b
    count += 1
print()
```

Expected:

```
0 1 1 2 3 5 8 13 21 34
```

The swap-and-add line is the same “evaluate the right-hand side first” idea as Names and Assignment.

break, continue, else

`break` leaves the loop immediately. `continue` skips the rest of *this* round and goes to the next item.

```
# first_even.py
def first_even(nums):
    for n in nums:
        if n % 2 == 0:
            return n
    return None

if __name__ == "__main__":
    print(first_even([1, 3, 4, 5]))
    print(first_even([1, 3, 5]))
```

Expected:

```
4
None
```

```
# for_else.py
for n in [1, 3, 5]:
    if n % 2 == 0:
        print("found", n)
        break
else:
    print("no even")
```

Expected:

```
no even
```

`for/while else` means “the loop finished without `break`.” It surprises people. Use it when the name in your head is “search failed.” A `return` inside the loop is often clearer than `else`.

```
# continue_evens.py
for n in range(5):
    if n % 2 == 1:
        continue
```

```
print(n)
```

Expected:

```
0
2
4
```

Looping with an index

```
# enumerate_words.py
words = ["ada", "grace", "alan"]
for i, word in enumerate(words, start=1):
    print(i, word)
```

Expected:

```
1 ada
2 grace
3 alan
```

Do not write `for i in range(len(words))` unless you truly need the index for assignment. `enumerate` is the usual spelling.

Nested loops

```
# grid.py
for row in range(1, 4):
    for col in range(1, 4):
        print(f"{row},{col}", end=" ")
    print()
```

Expected:

```
1,1 1,2 1,3
2,1 2,2 2,3
3,1 3,2 3,3
```

The inner loop runs completely for each outer value. When both ranges grow with the input, the inner body runs about n^2 times — time a small n and a larger n if you want to feel it.

Python + CS

A `for` over a collection is **iteration**. An object that works in `for` is an **iterable** (it can produce its items one at a time). Internals and Idioms make that precise. `while` is the raw “keep going while this is true” that `for` is built on.

Pitfalls

Mutating a list you are iterating skips items. Iterate a copy or build a new list:

```
# mutate_while_iterating.py
nums = [1, 2, 3, 4]
# bad: for x in nums: nums.remove(x)    # skips items
kept = [x for x in nums if x % 2 == 1]
print(kept)
```

Expected:

```
[1, 3]
```

- `while True:` without a `break` is a daemon loop — fine for a server, not for this chapter.
- `range(len(xs))` plus `xs[i]` is usually `for x in xs`.
- Off-by-one: `range(1, 10)` stops at 9. For 1 through 10 use `range(1, 11)`.

Exercises

1. Sum the integers from 1 to 100 two ways (you already have `sum_to_100.py`). Change the upper bound to 10 and predict first.
2. Save `fibonacci.py` and print the first 15 numbers.
3. Search for "q" in "python" with `for/else` and print "missing" if it is not there. Then do the same with `if "q" in s`.

Further reading

- `for`, `range`, `break`

Comprehensions

Comprehensions

After this page you can build a new list (or set, or dict) from an old one in one line — and you will know when a plain `for` is kinder.

A comprehension is a `for` loop that **returns** a collection.

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Mental model

```
[ expr for item in seq if cond ]
```

```
            iterate      keep some (optional)
transform each item
```

Same idea for `{...}` (set or dict) and `(...)` (generator — lazy, one value at a time).

Lists

```
# squares.py
xs = [1, 2, 3, 4]
squares = [x * x for x in xs]
evens = [x for x in xs if x % 2 == 0]
print(squares, evens)

built = []
for x in xs:
    built.append(x * x)
print(built)
```

Expected:

```
[1, 4, 9, 16] [2, 4]
[1, 4, 9, 16]
```

Prefer the comprehension when the body is a single expression. Prefer a loop when you need two statements, a `try`, or a comment in the middle.

Worked example: squares of odd numbers

```
# odds_squared.py
odds_squared = [n * n for n in range(20) if n % 2 == 1]
print(odds_squared)
print(odds_squared[:5])
```

Expected:

```
[1, 9, 25, 49, 81, 121, 169, 225, 289, 361]
[1, 9, 25, 49, 81]
```

Read it left to right: take `n` from `0..19`, keep the odd ones, square them.

Sets and dicts

```
# fold_words.py
words = ["Ada", "ada", "Grace"]
print({w.lower() for w in words})
print({w.lower(): len(w) for w in words})
```

Expected:

```
{'ada', 'grace'}
{'ada': 3, 'grace': 5}
```

(Set order may differ.) Curly braces with a single expression make a **set**. Curly braces with **key: value** make a **dict**.

Dict keys must be unique; a later pair overwrites an earlier one. That is why `"Ada"` and `"ada"` collapse to one key after `.lower()`.

Worked example: invert a dict

```
# invert.py
ages = {"a": 1, "b": 2}
inverted = {v: k for k, v in ages.items()}
print(inverted)
```

Expected:

```
{1: 'a', 2: 'b'}
```

If two keys shared a value, the last one would win. Invert only when values are unique, or decide a policy (list of keys, first wins, ...).

Nested

```
# pairs.py
pairs = [(r, c) for r in range(3) for c in range(3)]
print(pairs)
print(pairs[:4], "...")
```

Expected:

```
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
[(0, 0), (0, 1), (0, 2), (1, 0)] ...
```

Read left to right: outer **for** first, same order as nested loops. If it does not fit on one line comfortably, use loops.

Generators (preview)

```
# gen_squares.py
squares = (x * x for x in range(1_000_000))
print(next(squares), next(squares))
print(type(squares))
```

Expected:

```
0 1
<class 'generator'>
```

Parentheses make a **generator**: values are produced on demand, not all at once. Idioms and Internals will come back to this. Do not wrap a huge **range** in [...] unless you need all the values in memory.

1_000_000 is just 1000000; the underscores are for your eyes.

When not to

- Side effects (**print**, writing a file) — use a **for** loop.
- Several steps per item — use a loop or a named function.
- You already have **map/filter** and the team prefers them. Comprehensions are the usual default in this book.

Python + CS

A comprehension is a **map** (transform every item) and optional **filter** (keep some). That is the same idea as “apply this recipe to each thing, skip the ones I do not want.”

Pitfalls

Worked example: the `[] * 3` trap

```
# shared_rows.py
bad = [[]] * 3
bad[0].append(1)
print("bad ", bad)

good = [[] for _ in range(3)]
good[0].append(1)
print("good", good)
```

Expected:

```
bad  [[1], [1], [1]]
good [[1], [], []]
```

* on a list repeats *references*. A comprehension builds a new inner list each time. `_` is the usual name for “I do not use this loop variable.”

- A dict comprehension that uses a list as a key will fail (`TypeError`). Keys must be hashable (immutable, roughly).
- Over-nested comprehensions are a flexibility contest. The next reader always loses.

Exercises

1. Run `odds_squared.py`. Change `range(20)` to `range(10)` and predict first.
2. Run `invert.py` on `{ "a": 1, "b": 1 }`. Which key survives?
3. Run `shared_rows.py`. Draw the arrows for `bad` and `good`.

Further reading

- [List comprehensions](#)

Functions

def and Arguments

def and Arguments

After this page you can write a function with defaults, call it with names or positions, and avoid the mutable-default trap.

A function is a **named computation**. `def` creates a function object and binds it to a name.

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Mental model

```
call  c_to_f(20)
      |
      v
new frame: celsius → 20
      |
      v
body runs; return value comes back
      |
      v
frame discarded
```

Each call gets its own **frame** — its own set of names. Two calls of `c_to_f` do not share a `celsius`. Internals draws this with real objects; you only need “each call is a fresh workspace” for now.

Minimal example

```
# c_to_f.py
def c_to_f(celsius):
    """Convert Celsius to Fahrenheit."""
    return celsius * 9 / 5 + 32

if __name__ == "__main__":
    print(c_to_f(0), c_to_f(100))

uv run python c_to_f.py
```

Expected:

```
32.0 212.0
```

The string under **def** is a **docstring**. `help(c_to_f)` in the REPL shows it. Write one sentence for anything you will still be using next week.

```
# greet.py
def greet(name):
    return f"hello, {name}"

if __name__ == "__main__":
    print(greet("Ada"))
    print(greet)
```

Expected (the second line's 0x... will differ):

```
hello, Ada
<function greet at 0x...>
```

`greet` without `()` is the function object. `greet("Ada")` is a **call**.

Kinds of arguments

```
# clip.py
def clip(n, lo=0, hi=100):
    return max(lo, min(n, hi))

if __name__ == "__main__":
    print(clip(150))
    print(clip(150, hi=80))
    print(clip(n=-5, lo=-10, hi=10))
```

Expected:

```
100
80
-5
```

Kind	Example	Meaning
Positional	<code>clip(150)</code>	first argument fills <code>n</code>
Keyword	<code>clip(150, hi=80)</code>	fill <code>hi</code> by name; skip <code>lo</code>
Default	<code>lo=0</code>	used when the caller omits it

You can mix them: positional first, then keywords. `clip(hi=80, 150)` is a `SyntaxError`.

Worked example: defaults are bound once

Defaults are bound **once**, when `def` runs — not each call. Never use a mutable default (`def f(xs=[])`). Use `None` and create a new list in the body.

```
# mutable_default.py
def broken(item, bucket=[]):
    bucket.append(item)
    return bucket

def fixed(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket

if __name__ == "__main__":
    print("broken", broken("a"), broken("b"))
    print("fixed ", fixed("a"), fixed("b"))
```

Expected:

```
broken ['a', 'b'] ['a', 'b']
fixed  ['a'] ['b']
```

`broken("a")` and `broken("b")` print the **same** list twice at the end of the first line — that is the bug.

* and / (preview)

```
# connect.py
def connect(host, port, /, *, timeout=30):
    return host, port, timeout

if __name__ == "__main__":
    print(connect("localhost", 8000))
    print(connect("localhost", 8000, timeout=5))
    # print(connect(host="localhost", port=8000)) # TypeError: positional-only
```

Expected:


```
('localhost', 8000, 30)
('localhost', 8000, 5)
```

Parameters before / are positional-only (you cannot write `host=`). Parameters after * are keyword-only (you must write `timeout=`). You will see this in `stdlib` signatures (`help(max)`). You rarely need it in Path A.

Arbitrary arguments

```
# extra_args.py
def total(*nums):
    return sum(nums)

def label(**meta):
    return meta

def show(*args, **kwargs):
    print("args", args)
    print("kwargs", kwargs)

if __name__ == "__main__":
    print(total(1, 2, 3))
    print(label(src="lab", n=2))
    show(1, 2, color="red")
```

Expected:

```
6
{'src': 'lab', 'n': 2}
args (1, 2)
kwargs {'color': 'red'}
```

`*nums` is a **tuple** of extra positional args. `**meta` is a **dict** of extra keywords.

Return

- `return value` exits immediately and sends `value` back.
- A bare `return` or falling off the end returns `None`.
- You can return several values: `return a, b` — that is a tuple. Unpack at the call: `x, y = f()`.

```
# min_max.py
def min_max(xs):
    return min(xs), max(xs)

if __name__ == "__main__":
    lo, hi = min_max([3, 1, 4])
    print(lo, hi)
```

Expected:

```
1 4
```

Python + CS

Functions are the unit of **abstraction**: hide a recipe behind a name so the caller does not have to see the steps. Recursion (a function calling itself) can wait until frames feel comfortable — try this when you are ready:

```
# factorial.py
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

if __name__ == "__main__":
    print(factorial(5))
    print(factorial(0))
```

Expected:

```
120
1
```

Pitfalls

- `def f(xs=[])` — one list shared across calls (see `mutable_default.py`).
- Forgetting `return` and wondering why the function “returns nothing” (`None`).
- Mutating a list or dict the caller passed in, unless the name says so (`sort` vs `sorted`).
- `clip(150, 80)` sets `lo=80`, not `hi`. Use `clip(150, hi=80)` when you skip a middle parameter.

Exercises

1. Copy `clip.py` to `clamp.py`, rename the function, and print the three edges (below, inside, above).
2. Write `mean.py` with `mean(*nums)` that returns `None` if there are no arguments, otherwise the average. Print `mean(1, 2, 3)` and `mean()`.
3. Run `mutable_default.py`. Then add a third `broken("c")` call and watch the list grow.

Further reading

- [Defining functions](#)

Scope

Scope

After this page you can predict which `n` a function sees, and you can write a small nested function that remembers a value.

A name is looked up in a nest of environments. Python's rule is **LEGB**: Local, Enclosing, Global, Built-in.

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Mental model

```
built-ins    len, print, int, ...    (always there)
  ^
global       names at the top of the file
  ^
enclosing    a def around this def
  ^
local        names assigned in this function
```

Python starts at the inside and walks out. The first place that has the name wins. If none do: `NameError`.

You can *see* which `n` is which by reading the source. The caller's names do not leak in.

Minimal example

```
# apply_scale.py
scale = 10          # global (top of the file)

def apply(n):
    factor = 2       # local (assigned in this function)
    return n * factor * scale

if __name__ == "__main__":
    print(apply(3))
```

```
# print(factor) # NameError - factor only exists inside apply
```

Expected:

```
60
```

`apply` can *read* `scale` because it did not assign to `scale`. `n` and `factor` live only for the duration of the call.

Assignment makes a name local

```
# bump.py
n = 1

def bump():
    n = 2          # local n, does not touch the global
    return n

if __name__ == "__main__":
    print(bump(), n)
```

Expected:

```
2 1
```

The `n = 2` is a *different sticky note*. The global `n` is still 1.

If you *must* rebind a global (rarely), use `global n`. If you must rebind a name in an enclosing function, use `nonlocal`. Prefer returning a new value instead.

```
# counter.py
def counter():
    n = 0

    def tick():
        nonlocal n
        n += 1
        return n

    return tick

if __name__ == "__main__":
    t = counter()
```

```

    u = counter()
    print(t(), t())
    print(u())

```

Expected:

```

1 2
1

```

`tick` is defined *inside* `counter`. `nonlocal n` means “the `n` in `counter`, not a new local.” Each call to `counter()` gets its own `n` — `t` and `u` do not share.

Worked example: UnboundLocalError

```

# unbound_local.py
n = 1

def broken():
    n = n + 1      # UnboundLocalError
    return n

if __name__ == "__main__":
    try:
        broken()
    except UnboundLocalError as err:
        print(type(err).__name__, err)

```

Expected:

```

UnboundLocalError cannot access local variable 'n' where it is not associated with a value

```

(The exact message can vary slightly by version.) Assignment anywhere in the function makes `n` local. Then `n + 1` tries to *read* that local before it has a value.

Fix by returning a value:

```

# next_n.py
def next_n(n):
    return n + 1

if __name__ == "__main__":
    n = 1
    n = next_n(n)

```

```
print(n)
```

Expected:

```
2
```

That is the usual pattern. `global` is for the rare case where many functions must share one flag.

Built-ins

```
# hide_builtin.py
print("before", len("py"))

len = 3
print("hid len, now len is", len)
# print(len("py")) # TypeError: 'int' object is not callable

del len
print("after del", len("py"))
```

Expected:

```
before 2
hid len, now len is 3
after del 2
```

`dir(__builtins__)` is the full list. Shadowing `list` or `str` is a rite of passage and a bad afternoon. `del len` recovers the builtin.

Closures (preview)

A nested function that uses an enclosing name **closes over** it. `tick` above keeps `n` alive after `counter` returns. Decorators and callbacks are built on this. Idioms will return here.

```
# make_adder.py
def make_adder(k):
    def add(n):
        return n + k
    return add

if __name__ == "__main__":
    plus_10 = make_adder(10)
    print(plus_10(5))
```



```
print(make_adder(3)(4))
```

Expected:

```
15
7
```

`k` is not global and not local to `add`. It lives in the enclosing frame of `make_adder`. That is the “E” in LEGB.

Python + CS

LEGB is a **lexical** (static) scope rule: you can see which `n` is which by looking at where it is written, not at who called whom. Dynamic scope (the caller’s names leak in) is not how Python works.

Pitfalls

- `n += 1` at the top of a function without `global/nonlocal` is an `UnboundLocalError` if `n` also exists outside.
- Loops do not create a scope. `for i in xs: ...` leaves `i` bound after the loop.
- Comprehensions *do* have their own scope in Python 3 (`[i for i in xs]` does not leak `i`).
- Two calls to `counter()` are two counters.

```
# loop_leaks.py
for i in range(3):
    pass
print("loop left i =", i)

squares = [n * n for n in range(3)]
print("squares", squares)
# print(n) # NameError - comprehension does not leak n
```

Expected:

```
loop left i = 2
squares [0, 1, 4]
```

Exercises

1. Predict what `bump.py` prints, then run it.
2. Run `make_adder.py`. Add `plus_2 = make_adder(2)` and print `plus_2(10)`.
3. Run `unbound_local.py`, then use `next_n.py` as the fix instead of `global`.

Further reading

- [Python scopes and namespaces](#)
- Internals → [Names, namespaces, frames](#)

First-Class Functions

First-Class Functions

After this page you can pass a function into `sorted`, return a small function, and know when a `lambda` is enough.

Functions are objects. You can pass them, return them, put them in a list, and attach attributes. That is what “first-class” means: a function is a value, like 20 or “ada”.

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Mental model

```
def greet(name): ...  
  |  
  v  
name "greet"      function object  
                    |  
                    +-- __name__ "greet"  
                    +-- you can call it: greet("ada")  
                    +-- you can pass it: apply(greet, "ada")
```

`greet` is just a name. `other = greet` is two names on one function.

Pass a function

```
# apply_twice.py  
def apply_twice(fn, x):  
    return fn(fn(x))  
  
def inc(n):  
    return n + 1  
  
if __name__ == "__main__":  
    print(apply_twice(inc, 3))  
    print(apply_twice(str.upper, "py"))
```

Expected:

```
5
PY
```

You pass `inc`, not `inc()`. `inc` is the function; `inc()` would *call* it with no arguments and pass the result.

`sorted(key=...)`, `max(key=...)`, and `min(key=...)` are the `stdlib` version of this idea.

```
# sort_by_len.py
words = ["banana", "fig", "apple"]
print(sorted(words))
print(sorted(words, key=len))
```

Expected:

```
['apple', 'banana', 'fig']
['fig', 'apple', 'banana']
```

`key=len` means “compare items by `len(item)`.” You pass the function `len`.

Worked example: sort by the number after =

```
# sort_ids.py
rows = ["id=12", "id=3", "id=100"]

def id_number(row):
    return int(row.split("=")[1])

if __name__ == "__main__":
    print(sorted(rows, key=id_number))
    print(sorted(rows))
```

Expected:

```
['id=3', 'id=12', 'id=100']
['id=100', 'id=12', 'id=3']
```

Without `key`, `"id=100"` sorts before `"id=12"` because `"1" < "3"` as text.

Return a function

```
# multiply_by.py
def multiply_by(k):
    def inner(n):
        return n * k
    return inner

if __name__ == "__main__":
    triple = multiply_by(3)
    print(triple(4))
    print(multiply_by(10)(2))
```

Expected:

```
12
20
```

`multiply_by(3)` *returns* a function. That function remembers `k = 3` (a closure — see Scope).

lambda

```
# sort_by_last.py
words = ["banana", "fig", "apple"]
print(sorted(words, key=lambda w: w[-1]))
```

Expected:

```
['banana', 'apple', 'fig']
```

A `lambda` is an anonymous function whose body is **one expression**. The line above sorts by last letter (a, e, g).

```
# last_char.py
def last_char(w):
    return w[-1]

if __name__ == "__main__":
    also = lambda w: w[-1] # legal, but a def is clearer if you want a name
    print(last_char("fig"), also("fig"))
```

Expected:

```
g g
```

If you want a name, two lines, or a docstring, use `def`.

Small higher-order tools

```
# map_filter.py
print(list(map(str.upper, ["a", "b"])))
print(list(filter(lambda n: n % 2 == 0, range(6))))
print([s.upper() for s in ["a", "b"]])
print([n for n in range(6) if n % 2 == 0])
```

Expected:

```
['A', 'B']
[0, 2, 4]
['A', 'B']
[0, 2, 4]
```

A comprehension is often clearer than `map/filter`. This book defaults to comprehensions.

Python + CS

Once functions are values, you can write **combinators** (functions that take functions) and **strategies** (pass the step that varies). `sorted(..., key=...)` is the everyday case: the sort algorithm stays put; *what to compare* is the argument.

Pitfalls

`sorted(xs, key=len())` calls `len` and passes its result. Pass `len`.

Worked example: the late-binding loop

A `lambda` that closes over a loop variable `i` and is called *later* sees the **last** `i`. Bind it as a default:
`lambda i=i: i`.

```
# late_binding.py
fns = []
for i in range(3):
    fns.append(lambda: i)
print([fn() for fn in fns])
```

```
fns = []
for i in range(3):
    fns.append(lambda i=i: i)
print([fn() for fn in fns])
```

Expected:

```
[2, 2, 2]
[0, 1, 2]
```

Default arguments are bound when the `lambda` is *created*. That is the usual fix.

`callable(x)` is the test for “can I call this?”. `type(x) is function` is the wrong test (built-ins such as `len` are a different type).

Exercises

1. Run `sort_ids.py`. Then sort the same rows with a `lambda` instead of `id_number`.
2. Write `compose.py` so `compose(f, g)(x)` is `f(g(x))`. Check `compose(str.upper, str.strip)(" hi ")` prints HI.
3. Replace the `map` line in `map_filter.py` with a comprehension (already shown). Do the same for `filter`.

Further reading

- [More on defining functions](#)

Data Structures

Lists and Tuples

Lists and Tuples

After this page you can grow a list, unpack a pair, copy a list without the shallow-copy surprise, and pick list vs tuple.

A **list** is a mutable sequence (you can grow it, shrink it, overwrite slots). A **tuple** is an immutable sequence (fixed length). Both are ordered and can hold mixed types.

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Mental model

<code>list</code>	<code>[1, 2, 3]</code>	grow, shrink, overwrite slots
<code>tuple</code>	<code>(1, 2, 3)</code>	fixed length; can be a dict key if items are hashable

Index and slice work the same way as on strings. `xs[0]` is the first item. `xs[-1]` is the last. `xs[1:3]` is a new list of those items.

Lists

```
# list_basics.py
xs = [3, 1, 4]
xs.append(1)
xs[0] = 30
print(xs, xs[1:3], len(xs))

ys = xs[:]          # shallow copy - a new outer list
xs.append(99)
print("xs", xs)
print("ys", ys)
```

Expected:

```
[30, 1, 4, 1] [1, 4] 4
xs [30, 1, 4, 1, 99]
ys [30, 1, 4, 1]
```

Method	Role
<code>append</code> , <code>extend</code>	Add one item / add a sequence
<code>insert</code> , <code>pop</code> , <code>remove</code>	At a position / last (or i) / first match
<code>sort</code> , <code>reverse</code>	In place (return <code>None</code>)
<code>count</code> , <code>index</code>	Frequency / first position

`sorted(xs)` returns a **new** list. `xs.sort()` returns `None` and changes `xs`.

```
# sort_vs_sorted.py
xs = [3, 1, 4]
print("sorted", sorted(xs))
print("xs still", xs)
print("sort returns", xs.sort())
print("xs now", xs)
```

Expected:

```
sorted [1, 3, 4]
xs still [3, 1, 4]
sort returns None
xs now [1, 3, 4]
```

Worked example: stack with a list

```
# stack.py
stack = []
stack.append("a")
stack.append("b")
print(stack.pop())
print(stack)
```

Expected:

```
b
['a']
```

A list is a fine **stack**. A list is a **bad** queue (`pop(0)` copies the tail every time). For first-in-first-out use `collections.deque`:

```
# queue.py
from collections import deque

q = deque(["a", "b"])
q.append("c")
print(q.popleft())
```

```
print(list(q))
```

Expected:

```
a
['b', 'c']
```

Tuples

```
# point.py
pt = (2, 3)
# pt[0] = 0      # TypeError
x, y = pt
print(x + y)

one = (2,)      # trailing comma - (2) is just 2
empty = ()
print(one, empty, type(one), type((2)))
```

Expected:

```
5
(2,) () <class 'tuple'> <class 'int'>
```

(2) is just the number 2. The comma makes a tuple.

Tuples are for **positions that mean something** (x, y), for multiple return values, and for records you do not want to grow. Named tuples and dataclasses (Objects) are the next step.

```
# bounds.py
def bounds(xs):
    return min(xs), max(xs)

if __name__ == "__main__":
    print(bounds([3, 1, 4]))
    lo, hi = bounds([3, 1, 4])
    print(lo, hi)
```

Expected:

```
(1, 4)
1 4
```

Parentheses are often optional; the **comma** makes a tuple. 2, 3 is already a tuple.

Shallow copies

```
# shallow.py
grid = [[0, 0], [0, 0]]
also = grid[:]
also[0][0] = 1
print("grid", grid)
print("also", also)
```

Expected:

```
grid [[1, 0], [0, 0]]
also [[1, 0], [0, 0]]
```

`[:]` copies the *outer* list. The inner lists are shared. `copy.deepcopy` copies the whole tree.

Worked example: a 3×3 grid of independent rows

```
# grid3.py
bad = [[0] * 3] * 3
bad[0][0] = 1
print("bad ", bad)

good = [[0] * 3 for _ in range(3)]
good[0][0] = 1
print("good", good)
```

Expected:

```
bad  [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
good [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

`[0] * 3` is fine (ints never change). `[[]] * 3` and `[[0] * 3] * 3` repeat *references*. A comprehension builds a new row each time.

Python + CS

Lists are the default **dynamic array** (a sequence that can grow). Tuples are the default **product type** (an x-and-y, a pair of results). Most later programs start with one of these two.

Pitfalls

```
# list_pitfalls.py
xs = ys = []
xs.append(1)
print("shared", ys)

a = [1]
print("append returns", a.append(2))
print("a is", a)

try:
    sorted([1, "a"])
except TypeError as err:
    print(type(err).__name__)
```

Expected:

```
shared [1]
append returns None
a is [1, 2]
TypeError
```

- `xs += [1]` mutates if `xs` is a list; `xs = xs + [1]` rebinds.
- Do not write `xs = xs.append(1)`.

Exercises

1. Reverse a list in place (`reverse`) and again with a slice (`xs[::-1]`). Confirm they are different objects in the second case (`xs[::-1] is xs` is `False`).
2. Write `mid.py` that returns the middle item of a non-empty sequence. Decide what “middle” means for even length and print both cases.
3. Run `grid3.py`. Then set `good[1][0] = 9` and print again — only that row should change.

Further reading

- [More on lists](#)
- [Tuples and sequences](#)

Dicts and Sets

Dicts and Sets

After this page you can look up a value by name, count things with a dict, and test membership with a set.

A **dict** maps hashable keys to values. A **set** is a dict that only keeps keys. Both are typically instant for add and lookup, even when they get large.

Hashable (roughly): immutable and comparable — **int**, **str**, **tuple** of hashables. Not **list**, **dict**, or **set**.

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Mental model

```
dict    key      value
        "ada"    1815

set     key      (present)
        "ada"
```

CPython 3.7+ dicts **remember insertion order**. Do not rely on order in a set.

Dicts

```
# born.py
born = {"ada": 1815, "grace": 1906}
born["alan"] = 1912
print(born["ada"])
print(born.get("nope", 0))
print(list(born))
print(list(born.values()))
for name, year in born.items():
    print(name, year)
```

Expected:

```
1815
0
```

```
['ada', 'grace', 'alan']
[1815, 1906, 1912]
ada 1815
grace 1906
alan 1912
```

Spelling	Role
<code>d[k]</code>	Get or set; missing key $\rightarrow$ <code>KeyError</code>
<code>d.get(k, default)</code>	Safe get
<code>d.setdefault(k, v)</code>	Get or insert <code>v</code>
<code>d.pop(k)</code>	Remove and return
<code>k in d</code>	Membership on keys

Worked example: count letters

```
# count_letters.py
from collections import Counter

counts = {}
for ch in "banana":
    counts[ch] = counts.get(ch, 0) + 1
print(counts)
print(Counter("banana"))
```

Expected:

```
{'b': 1, 'a': 3, 'n': 2}
Counter({'a': 3, 'n': 2, 'b': 1})
```

`d[k] += 1` fails if `k` is missing (`KeyError`). `.get(ch, 0)` supplies a zero for the first sighting. Use `Counter` when you are actually counting.

Sets

```
# seen.py
seen = {"ada", "grace"}
seen.add("alan")
print("ada" in seen)
print(seen & {"ada", "linus"})
print(seen | {"linus"})
print(seen - {"grace"})
print(set([1, 1, 2, 3]))
print(type({}), type(set()))
```

Expected (set order may differ):

```
True
{'ada'}
{'ada', 'grace', 'alan', 'linus'}
{'ada', 'alan'}
{1, 2, 3}
<class 'dict'> <class 'set'>
```

Empty set is `set()`, not `{}` — `{}` is an empty **dict**. Sets drop duplicates. They are the right type for “have I already seen this?”

Worked example: names in both lists

```
# both_names.py
lab = ["ada", "grace", "alan"]
class_list = ["grace", "linus", "ada"]
both = set(lab) & set(class_list)
print(both)
print([name for name in lab if name in set(class_list)])
```

Expected (set order may differ on the first line):

```
{'grace', 'ada'}
['ada', 'grace']
```

Converting to sets makes membership cheap and drops duplicates. The list comprehension keeps the original order of `lab`.

What can be a key

```
# hashable_key.py
ok = {("ada", 1815): "ok"}
print(ok[("ada", 1815)])
try:
    bad = {"ada": "no"}
except TypeError as err:
    print(type(err).__name__, err)
```

Expected:

```
ok
TypeError unhashable type: 'list'
```

If you mutate an object that is already a key, the dict cannot find it again. That is why only immutables are allowed.

dict vs list vs set — pick

Question	Type
Ordered collection, duplicates ok	<code>list</code>
Fixed record / pair	<code>tuple</code>
Lookup by name	<code>dict</code>
Membership only	<code>set</code>

If you catch yourself writing `for row in rows: if row[0] == name`, you probably wanted a dict keyed by `name`.

Python + CS / maths

A dict is a **hash table**: turn the key into a number, jump to a slot. A set's `&` `|` `-` are operations you can try on two small sets — Python showing membership, not a lecture on relations.

```
# set_ops.py
a = {1, 2, 3}
b = {3, 4}
print("intersection", a & b)
print("union      ", a | b)
print("difference ", a - b)
```

Expected:

```
intersection {3}
union        {1, 2, 3, 4}
difference   {1, 2}
```

CPython's dict is also how objects store attributes (Internals). That is why `obj.x` and `d["x"]` feel similar.

Pitfalls

```
# missing_key.py
d = {}
try:
    d["a"] += 1
except KeyError:
```

```
d["a"] = d.get("a", 0) + 1
print(d)
```

Expected:

```
{'a': 1}
```

- `if key in d.values()` is a linear scan. Membership is for keys.
- Building a set of lists: convert each list to a tuple first.
- Do not write tests that depend on set order.

Exercises

1. Run `count_letters.py` on "mississippi".
2. Given two lists, print the names that appear in both (start from `both_names.py`).
3. Invert `{ "a": 1, "b": 2 }` to `{ 1: "a", 2: "b" }`. What should happen if values collide? Decide, then write `invert.py`.

Further reading

- [Dictionaries](#)
- [Sets](#)
- Internals → [Object model](#)

Unpacking

Unpacking

After this page you can split a pair into names, grab “the rest” with `*`, and pass a list or dict into a function with `*` / `**`.

Unpacking assigns **parts of a structure** to names. It is how Python spells “this sequence has this shape.”

Every `python` block below is a complete file. The first line is the filename. Save it and run `uv run python that_file.py`.

Mental model

```
left      right
a, b      =   (1, 2)
a, *rest  =   [1, 2, 3]
{k: v}    in  a dict merge or match
```

`match` (Control flow) is unpacking plus branching.

Sequences

```
# unpack_seq.py
x, y = (2, 3)
head, *mid, tail = [1, 2, 3, 4]
first, second = "ab"
print(x, y)
print(head, mid, tail)
print(first, second)
```

Expected:

```
2 3
1 [2, 3] 4
a b
```

The number of names must match, unless you use `*`. Too many or too few values: `ValueError`.

```
# unpack_error.py
try:
    a, b = [1, 2, 3]
except ValueError as err:
    print(type(err).__name__, err)
```

Expected:

```
ValueError too many values to unpack (expected 2)
```

*mid soaks up whatever is left. It is always a **list**, even if you unpacked a tuple.

Worked example: first and last name

```
# names.py
first, last = "ada lovelace".split()
print(first, last)

first, *rest = "ada king lovelace".split()
print(first, rest)
```

Expected:

```
ada lovelace
ada ['king', 'lovelace']
```

This fails (on purpose) if the first line's string has one word or three. That is often what you want.

Swapping and returning

```
# swap_bounds.py
a, b = "tea", "coffee"
a, b = b, a
print(a, b)

def bounds(xs):
    return min(xs), max(xs)

if __name__ == "__main__":
    lo, hi = bounds([3, 1, 4])
    print(lo, hi)
```

Expected:


```
coffee tea
1 4
```

The right-hand side is evaluated fully, then the left is bound. That is why swap works without a temporary.

Stars in calls

```
# star_call.py
def clip(n, lo, hi):
    return max(lo, min(n, hi))

if __name__ == "__main__":
    args = (150, 0, 100)
    print(clip(*args))
    opts = {"n": 150, "lo": 0, "hi": 100}
    print(clip(**opts))
    try:
        clip(*[150, 0])
    except TypeError as err:
        print(type(err).__name__, err)
```

Expected:

```
100
100
TypeError clip() missing 1 required positional argument: 'hi'
```

* expands a sequence into positional arguments. ** expands a dict into keywords. The keys must match parameter names.

Merging (3.9+)

```
# merge_dicts.py
defaults = {"host": "localhost", "port": 8000}
user = {"port": 9000}
print("right wins", defaults | user)
print("left wins ", user | defaults)
print("old spell ", {**defaults, **user})
print("lists      ", [1, 2] + [3], [*[1, 2], *[3]])
```

Expected:

```

right wins {'host': 'localhost', 'port': 9000}
left wins  {'port': 8000, 'host': 'localhost'}
old spell  {'host': 'localhost', 'port': 9000}
lists      [1, 2, 3] [1, 2, 3]

```

| on dicts is a merge. The **right-hand** dict wins on conflicts. `**defaults, **user` is the older spelling. For lists, `*xs, *ys` or `xs + ys`.

Nested

```

# nested.py
pt, color = ((2, 3), "red")
(x, y), color = ((2, 3), "red")
print(pt, color)
print(x, y, color)

```

Expected:

```

(2, 3) red
2 3 red

```

If the nesting is deeper than one level, `match` or a named tuple is usually kinder.

Pitfalls

```

# unpack_dict.py
d = {"x": 1, "y": 2}
a, b = d
print("keys", a, b)
(ka, va), (kb, vb) = d.items()
print("pairs", ka, va, kb, vb)

parts = "ada".split()
try:
    first, last = parts
except ValueError as err:
    print(type(err).__name__, err)

```

Expected:

```

keys x y
pairs x 1 y 2
ValueError not enough values to unpack (expected 2, got 1)

```

- `*rest` is always a **list**, even if you unpacked a tuple.
- `def f(*args, **kwargs)` then `f(*args, **kwargs)` is the usual wrap; missing `*` / `**` passes one tuple and one dict as two positional args.
- Guard with `if len(parts) == 2` when input is messy.

Exercises

1. Run `names.py`. Then try `"ada"` alone and read the `ValueError`.
2. Write `tail.py` using `_`, `*rest` that works for one or more items. Print what an empty list does.
3. Run `merge_dicts.py` and change the two dicts so three keys overlap. Confirm which side wins.

Further reading

- [Tuples and sequences](#)
- [Unpacking argument lists](#)

Modules and Packages

Intro

Modules and Packages

A **module** is a `.py` file. A **package** is a directory of modules. `import` binds names; `uv / pyproject.toml` is how third-party modules arrive.

Planned chapters

Chapter	Focus
Import and <code>sys.path</code>	What <code>import</code> <code>util</code> searches
Packages and <code>__init__</code>	Layout, relative imports
<code>uv add / lock / sync</code>	Dependencies as a file, not a snowflake machine
Scripts vs packages	<code>__main__.py</code> , entry points
Publishing (later)	Build a wheel when you need one

Until these leaves exist: follow [Project layout](#) and the official [modules](#) + [venv](#) tutorial chapters.

See also

- Internals → [Import system](#)

Objects and Classes

Intro

Objects and Classes

A **class** is a type you write. An **instance** is an object of that type. Methods are functions that receive the instance (**self**).

Planned chapters

Chapter	Focus
<code>class</code> and <code>self</code>	Instances, <code>__init__</code>
Dunder basics	<code>__repr__</code> , <code>__eq__</code> , <code>__len__</code>
Inheritance and <code>super</code>	When to compose instead
Dataclasses	Records without boilerplate
Protocols / ABC	“Looks like” vs “is a” (with Typing)

Until these leaves exist: official [Classes](#) tutorial, then Internals → [Object model](#) and [Descriptors](#).

Ramalho’s *Fluent Python* is the deep track for the data model.

Errors and I/O

Intro

Errors and I/O

Failures are **exceptions**. Files and other resources are **context managers** (**with**). Paths are **pathlib.Path**, not string concatenation.

Planned chapters

Chapter	Focus
<code>try / except / raise</code>	Catch what you can handle; let the rest go
Exception groups (3.11+)	Several failures at once
<code>pathlib</code>	Read, write, walk a tree
<code>with</code> and your own context managers	open , locks, sessions
Text vs bytes	Encoding at the edge

Until these leaves exist: official [Errors](#) and [Input and Output](#).

See also

- Standard Library → `pathlib` / `os` (same tools, as a map)

Stdlib Essentials

Intro

Stdlib Essentials

Reach for the standard library before a framework. This section is the *working set*; **Standard Library** (part 98) is the map.

Planned chapters

Module	Job
<code>pathlib, os</code>	Files and environment
<code>json, csv</code>	Structured text
<code>datetime / zoneinfo</code>	Time
<code>argparse</code>	Command-line flags
<code>logging</code>	What happened, when
<code>re</code>	Patterns (use sparingly)
<code>collections, itertools</code>	Batteries for loops
<code>subprocess</code>	Run a command

Official tours: [stdlib 1](#) and [stdlib 2](#).

Typing

Intro

Typing

Type hints are **optional** and checked by tools (`ty`, `Pyright`, `mypy`), not by CPython at runtime (except where a library such as `Pydantic` reads them).

Python 3.14 **defers annotation evaluation** (PEP 649 / 749): you can write `def f() -> Node:` before `Node` exists more often than in 3.13.

Planned chapters

Chapter	Focus
Built-in hints	<code>list[int]</code> , <code>dict[str, int]</code> , unions
<code>Protocol</code>	Structural typing
Generics	type parameters
<code>TypedDict</code> / <code>dataclasses</code>	Records
Checkers	<code>ty</code> , <code>Pyright</code> , <code>mypy</code> — pick one and run it in CI
<code>Pydantic v2</code>	Runtime validation when you parse untrusted input

Viafore, *Robust Python*, and Ramalho’s typing chapters are the book-length tracks. Do not start Path A here.

Testing

Intro

Testing

A test is a program that **fails when the behavior you care about changes**. This book's default is `pytest`.

Planned chapters

Chapter	Focus
First pytest	<code>assert</code> , discovery, <code>uv add --dev pytest</code>
Fixtures	Setup without copy-paste
Parametrize	Many examples, one test
Hypothesis	Property tests
Testing I/O	<code>tmp_path</code> , mocks as a last resort

Until these leaves exist: write `assert c_to_f(0) == 32` in `test_celsius.py` and run `uv run pytest`. Slatkin's *Effective Python* has collaboration/testing items; the `pytest` docs are the reference.

Idioms

Intro

Idioms

Idiomatic Python is the intersection of **PEP 8**, the data model, and twenty years of “we tried the other way.”

Planned chapters

Chapter	Focus
PEP 8 + Ruff	One style, automated
Iterators and generators	<code>yield</code> , <code>itertools</code>
Decorators	Functions that wrap functions
Context managers	<code>with</code> you write
EAFP vs LBYL	<code>try</code> vs pre-checks
Effective / Fluent highlights	A short item list, original examples

Sources we distill (not copy): Slatkin *Effective Python* 2e, Ramalho *Fluent Python* 2e, Beazley *Python Distilled*.

Concurrency

Intro

Concurrency

Three tools, three jobs:

Tool	Job
<code>threading</code>	Overlap waiting (I/O) in one process
<code>multiprocessing</code>	Overlap CPU in several processes
<code>asyncio</code>	Many I/O waits on one thread (event loop)

The **GIL** (Internals) explains why CPU-bound threads do not scale on the default build. Free-threaded 3.14 is a fourth option you measure, not a default.

Planned chapters

Chapter	Focus
Threads and Lock	Shared memory, invariants
Processes and queues	Isolation, pickle boundaries
<code>concurrent.futures</code>	The usual facade
<code>asyncio</code> primer	<code>async def</code> , <code>await</code> , tasks
When to pick which	A decision table

Ramalho's concurrency parts and *High Performance Python* are the deep books. This section is the language view (threads, processes, `asyncio`).

Internals

Intro

Internals

This section is the **machine** under the notation. It is Path B. You should already be able to write a small module, a function, and a test. If not, finish Getting started through Data structures first — this book is self-contained; come back here later.

Mental model

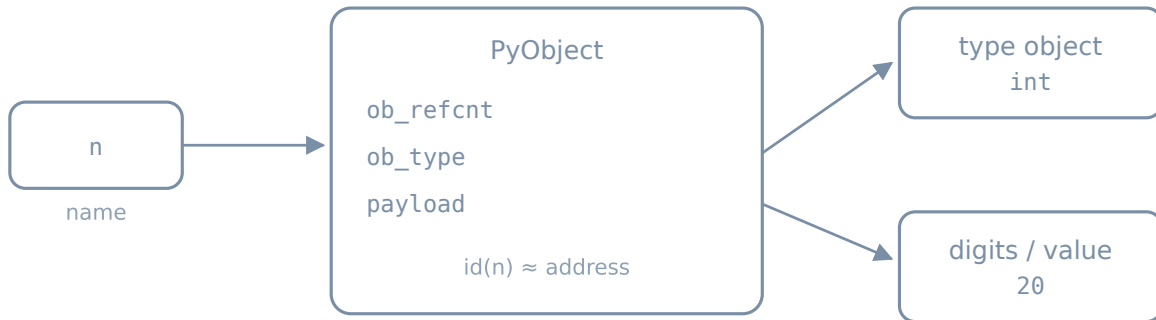
```
your .py file
  |
  v
parse → AST → compile → code object (bytecode)
                        |
                        v
                    eval loop (CEVAL)
                        |
                        v
                PyObject heap + frames
```

CPython is a C program that walks that pipeline. We stay at the level you can see from Python (`dis`, `sys.getsizeof`, `gc`, `__code__`) and only dip into C names when they are the usual ones (`PyObject`, `GIL`).

Chapters

Page	Question it answers
Object model	What is an object? type, id, refcount
Names, namespaces, frames	Where do names live at runtime?
Bytecode and eval	What does <code>dis</code> print?
GIL and free-threading	Why threads and CPU, and what 3.14 changed
Memory and GC	Refcount + cyclic collector
Import system	How <code>import</code> finds a module
Descriptors and slots	How <code>obj.x</code> actually works

A name is a label on an object



`type(n)` follows `ob_type`. Rebinding `n` moves the label; the object stays until refcount hits 0.
Free-threaded CPython keeps the language; the header's counting story changes.

Figure 1: Object model

How to work these pages

1. Run every `dis` / `sys` snippet.
2. Draw the objects. If you cannot draw it, you do not have it yet.
3. Stay at the Python-visible layer (`dis`, `sys`, `__code__`) unless a later internals page opens a C name on purpose. This book will not become a C tutorial.

What this is not

- Not a substitute for Shaw's *CPython Internals* or the [devguide](#).
- Not a JIT textbook. 3.13+ has an experimental JIT and 3.14 a faster eval loop; we treat them as “the machine got faster,” not as something you configure on day one.
- Not PyPy or MicroPython internals.

Further reading

- [CPython internals \(devguide\)](#)
- Anthony Shaw, *CPython Internals*
- Luciano Ramalho, *Fluent Python* 2e — data model chapters
- [What's New in Python 3.14](#)

Object Model

Object Model

Everything you can put in a name is an **object**: a chunk of memory with a type, a value, and a reference count (on the default CPython build).

Mental model

```
name "n"      PyObject
               +-- ob_refcnt
               +-- ob_type      type object (int, list, function, ...)
               +-- payload      (the digits, the items, the bytecode, ...)
```

`type(n)` follows `ob_type`. `id(n)` is, on CPython, the address of that struct. `sys.getrefcount` reads `ob_refcnt` (plus a temporary reference from the call itself).

Every python block below is a complete file. Save it and run `uv run python that_file.py`.

Minimal example

```
# object_header.py
import sys

n = 20
print(type(n), id(n))
print(sys.getrefcount(n))  # at least 2: name n + the argument
print(n.__class__ is type(n))
```

Small integers are **interned** (cached). `id(20)` may match `id(20)` from another line. That is an implementation detail; do not write code that needs it.

Type objects are objects

```
# types_are_objects.py
print(type(20) is int)
print(type(int) is type)
print(type(type) is type)
```

```

class Point:
    pass

p = Point()
print(type(p) is Point)
print(isinstance(p, Point))

```

Expected (the Point lines):

```

True
True
True
True
True

```

`int` is an object of type `type`. `type` is an object of type `type`. That loop is how a dynamic language describes itself. A **class** you write is just another type object.

Identity vs equality

```

# identity.py
a = [1]
b = [1]
print(a == b, a is b)

c = a
print(a is c)
one = 1
print(True == one, True is one)

```

Expected:

```

True False
True
True False

```

`==` asks the type's equality (`__eq__`). `is` asks "same address?" Use `is` for `None` and other singletons, not for numbers or strings (interning will lie to you in small cases).

Reference counts (default build)

```
# refcount.py
import sys

xs = []
print(sys.getrefcount(xs))
ys = xs
print(sys.getrefcount(xs))
del ys
print(sys.getrefcount(xs))
```

When the count hits zero, CPython frees the object immediately. Cycles (`a.x = a`) need the cyclic GC — [Memory and GC](#).

On a **free-threaded** build, biased/deferred reference counting replaces the simple picture. The language does not change; `sys.getrefcount` may be an estimate.

Everything has a type, including functions

```
# function_object.py
def f():
    return 1

if __name__ == "__main__":
    print(type(f), f.__code__.co_argcount)
    print(f())
```

A function object points at a **code object** (bytecode + constants). [Bytecode and eval](#) opens that box.

CS connection

This is a **tagged heap**: values know their type at runtime. In C you track the type; in a static language the compiler does. Dynamic typing is this object header, not magic.

Pitfalls

- `sys.getrefcount` is easy to misread (the argument adds a ref).
- id reuse: after an object dies, a new one may get the same address. Do not store ids as durable keys.
- `type(x) == list` misses subclasses; `isinstance(x, list)` is the check.

Exercises

1. Draw the objects for `a = []`; `b = a`; `c = []`.
2. Show that `True` is not `1` and `True == 1`.
3. Create a tiny class, instance it, and print `instance.__class__.__name__`.

Further reading

- [Data model](#)
- Ramalho, *Fluent Python* 2e, chapter 1
- Shaw, *CPython Internals* — object header

Names, Namespaces, Frames

Names, Namespaces, and Frames

A **namespace** is a mapping from names to objects (usually a dict). A **frame** is a namespace plus a position in a code object — one in-flight call.

Mental model

```
call f(3)
  |
  v
frame
  f_code    → f.__code__
  f_locals  → { "n": 3, ... }
  f_globals → the module dict
  f_back    → the caller's frame
```

`locals()`, `globals()`, and `sys._getframe()` (debug only) expose this.

Every python block below is a complete file. Save it and run `uv run python that_file.py`.

Minimal example

```
# frames.py
x = "module"

def f(n):
    y = n + 1
    print("locals", locals())
    print("x from here", x)
    return y

if __name__ == "__main__":
    f(3)
    print("globals has x", "x" in globals())
```

`x` is found in the global namespace. `n` and `y` live in the frame's locals.

LEGB at runtime

Letter	Runtime place
L	Current frame's locals (an array, not always a dict, until you ask)
E	Cell / free variables — closures
G	Module <code>__dict__</code>
B	<code>builtins</code> module

CPython stores fast locals in an array on the frame for speed. `locals()` builds a dict when you call it. Changing that dict may not write back. Treat `locals()` as read-only.

Closures are cells

```
# cells.py
def make_adder(k):
    def add(n):
        return n + k
    return add

if __name__ == "__main__":
    add2 = make_adder(2)
    print(add2(3))
    print(add2.__closure__[0].cell_contents)
```

Expected:

```
5
2
```

`k` is a **cell**: a tiny object that both `make_adder` and `add` point at. That is how `k` survives after `make_adder` returns.

Modules are dicts

```
# module_dict.py
import math

print(math.__dict__["pi"] is math.pi)
```

`import math` binds the name `math` in *your* module dict to the `math` module object. The module object has its own dict. [Import system](#) is how that object is found.

Stack traces are frame chains

```
# frames_traceback.py
import traceback

def inner():
    raise RuntimeError("boom")

def outer():
    inner()

if __name__ == "__main__":
    try:
        outer()
    except RuntimeError:
        traceback.print_exc()
```

Each line in the traceback is a frame. Recursion depth is “how long this chain is allowed to get” (`sys.getrecursionlimit()`).

CS connection

An interpreter is a **stack of environments**. This is the same picture as the CEK machine or a textbook “activation record.” You now have it in a language you can poke.

Pitfalls

- Assigning in a function makes a name local for the *whole* function (compiler decision), which is why `n += 1` can `UnboundLocalError`.
- `global` / `nonlocal` are compiler directives, not runtime searches.
- `sys._getframe()` is for debuggers. Do not use it in library code.

Exercises

1. Print `globals().keys()` in a script and name three that you did not put there (`__name__`, ...).
2. Build a closure and read `cell_contents`.
3. Write two nested calls and print `sys._getframe().f_back.f_code.co_name` from the inner one.

Further reading

- [Execution model](#)
- [inspect](#) for a supported way to look at frames

Bytecode and Eval

Bytecode and Eval

CPython does not run your source line by line. It **compiles** a code object, then an eval loop **interprets** the bytecode.

Mental model

```
source
|
v
parse (AST)
|
v
compile → code object
           co_code      bytes
           co_consts    20, "hello", ...
           co_varnames  local names
           co_names     globals / attrs
|
v
eval loop
  fetch opcode → execute → next
  (3.11+ specializing adaptive interpreter;
   3.14 also has a faster tail-call eval loop)
```

You do not configure this. You **read** it with `dis` when a function is surprising.

Every python block below is a complete file. Save it and run `uv run python that_file.py`.

Minimal example

```
# dis_add.py
import dis

def add(a, b):
    return a + b
```

```

if __name__ == "__main__":
    dis.dis(add)
    print(add.__code__.co_varnames, add.__code__.co_consts)
    print(add(2, 3))

```

You will see opcodes such as `RESUME`, `LOAD_FAST`, `BINARY_OP`, `RETURN_VALUE`. Names change between minor versions. Read the *story*: `load a`, `load b`, `add`, `return` — not a particular hex listing from 3.11.

compile and eval

```

# compile_eval.py
import dis

code = compile("2 + 3", "<demo>", "eval")
print(eval(code))
dis.dis(code)

```

`compile(source, filename, mode)` with mode `"eval"` (expression), `"exec"` (statements), or `"single"` (REPL). `eval` runs an expression; `exec` runs statements. Both are powerful and unsafe on untrusted strings.

Why this matters

```

# plus_vs_iadd.py
import dis

def loop_plus(xs):
    t = 0
    for x in xs:
        t = t + x
    return t

def loop_iadd(xs):
    t = 0
    for x in xs:
        t += x
    return t

if __name__ == "__main__":

```

```

print(loop_plus([1, 2, 3]), loop_iadd([1, 2, 3]))
print("--- + ---")
dis.dis(loop_plus)
print("--- += ---")
dis.dis(loop_iadd)

```

For integers these are almost the same. For a list, `t = t + [x]` copies; `t += [x]` mutates. `dis` shows `BINARY_OP(+)` versus `+=` in situ. Measure with `timeit` before you care; Internals is so you can *see* the difference.

Specialization (3.11+)

The adaptive interpreter rewrites some opcodes after they have seen the same types a few times (+ on two ints becomes a fast path). That is why a microbenchmark’s first milliseconds can differ from the rest. You still write normal Python.

The 3.13+ **JIT** (experimental) and 3.14 **tail-call interpreter** sit under this loop. They are CPython performance work, not a new language.

CS connection

Bytecode is a **stack machine** IR: load, load, binary op, store. This page is “enough to debug,” not a compilers course.

Pitfalls

- Comparing `dis` output across Python versions as if opcodes were a public API.
- `eval` on network input.
- Optimizing bytecode by hand. Change the algorithm or the data layout instead.

Exercises

1. `dis.dis` a function that builds a list with `append` and one that uses a comprehension. Note one difference.
2. compile a tiny `exec` block that binds `x = 1` and run it with `exec` into a dict you provide (`exec(code, ns)`).
3. Time `sum(range(10_000))` versus a Python `for` loop with `timeit`. The builtin wins; say why (C loop vs bytecode loop).

Further reading

- [dis](#)
- [What's New — specializing interpreter](#)
- Shaw, *CPython Internals* — compiler and CEVAL

GIL and Free-Threading

GIL and Free-Threading

The **Global Interpreter Lock** is a CPython mutex: in the **default** build, only one thread runs Python bytecode at a time. **Python 3.14** made a **free-threaded** (no-GIL) build official. Same language, different machine.

Mental model

```
default 3.14
  thread A          ( = holds GIL, runs bytecode)
  thread B
  C code / I/O can release the GIL

free-threaded 3.14
  thread A
  thread B          both run bytecode
  memory model and some extensions must cope
```

What the GIL does and does not do

True	False
One bytecode-running thread per process (default build)	“Python has no threads”
I/O and many C extensions release the GIL	Threads never help
<code>threading</code> is still useful for I/O wait	CPU-bound Python threads scale on 8 cores (default build)

For CPU-bound work on the default build, use **multiprocessing**, NumPy/PyTorch (C loops), or a free-threaded interpreter you have actually tested.

See it

```
# burn_threads.py
import threading
import time

def burn(n):
    x = 0
    for i in range(n):
        x += i
    return x

def run(workers):
    t0 = time.perf_counter()
    threads = [threading.Thread(target=burn, args=(4_000_000,)) for _ in range(workers)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    return time.perf_counter() - t0

if __name__ == "__main__":
    print("1 thread", run(1))
    print("4 threads", run(4))
```

On a default GIL build, 4 threads of this loop are **not** 4× faster (often similar or slower). On a free-threaded build, they can scale. I/O-bound threads (waiting on sockets) overlap even with the GIL.

3.13–3.14 timeline

Release	Free-threading
3.13	Experimental opt-in build (PEP 703)
3.14	Official (PEP 779); still a separate build / ABI

`uv python install 3.14` gives you the GIL build unless you ask for a free-threaded interpreter. Some binary wheels lag behind.

Check:

```
# gil_status.py
import sys

print(sys.version)
print(getattr(sys, "_is_gil_enabled", lambda: "n/a")())
```

(API names around GIL detection have been moving; read `help(sys)` on your exact 3.14 patch.)

Multiple interpreters (PEP 734)

3.14 also exposes **subinterpreters** in the `stdlib`: separate isolated states inside one process, each historically with its own GIL. They are a third isolation tool next to threads and processes. Treat them as advanced until you have a reason.

When to care

- Writing a web worker that waits on HTTP: threads or **asyncio** (Concurrency section). GIL is not your problem.
- Burning CPU in pure Python: measure; then processes, native code, or a tested free-threaded build.
- Shipping a C extension: know whether you support the free-threaded ABI.

CS connection

A GIL is a **coarse lock** on the interpreter. Free-threading replaces it with finer safety (atomics, biased refcounting). That is a runtime problem, not a language-syntax problem.

Pitfalls

- Switching production to free-threading because a blog said “no GIL.” Measure, test extensions, read the porting notes.
- Using **threads** as a synonym for “uses all cores.”
- Sharing a non-thread-safe list across threads and blaming the GIL when you forgot a **Lock**. The GIL is not a substitute for your own locks on *invariants*.

Exercises

1. Run the **burn** snippet with 1 and 4 threads. Record the times and your `sys.version`.
2. Change **burn** to `time.sleep(0.5)` (I/O-like wait). Compare 1 vs 4 threads again.
3. In one paragraph, explain why NumPy matrix multiply can use multiple cores even on a GIL build.

Further reading

- [PEP 703](#) — making the GIL optional
- [PEP 779](#) — free-threaded CPython official
- [PEP 734](#) — multiple interpreters
- Concurrency section (language view) once it is written; until then: [threading](#)

Memory and GC

Memory and GC

CPython’s default collector is **reference counting** plus a **cyclic garbage collector** for objects that point at each other.

Mental model

```
refcount → 0      free now
cycle (a→b→a)     refcounts never hit 0 → gc module walks generations
```

What to use today

```
# sizeof.py
import gc
import sys

print(gc.get_threshold())
print(sys.getsizeof([0] * 100))
print(sys.getsizeof("python"))
```

`sys.getsizeof` is the object header plus some payload — not the size of objects it *points to*. A list of large strings is bigger than `getsizeof(list)`.

Planned depth (later)

- Generations and `gc.collect()`
- `__del__` vs `weakref.finalize` (do not write `__del__` unless you must)
- Allocator: `pymalloc`, arenas
- Free-threaded refcounting (biased / deferred)

Until those pages exist: treat “cycles need gc” and “measure with `tracemalloc` / a profiler, not guesses” as the working rules.

Further reading

- [gc](#)
- [sys.getsizeof](#)
- Shaw, *CPython Internals* — memory management

Import System

Import System

`import math` asks a pipeline to **find** a module, **load** it once, and **bind** a name in the current namespace.

Mental model

```
import math
|
v
sys.modules cache?  yes → reuse
|
no
v
finders (sys.meta_path) → loader → module object
|
v
sys.modules["math"] = module
bind name "math" in the caller
```

What to use today

```
# import_math.py
import math
import sys

print(getattr(math, "__file__", None)) # None for some builtins
print("math" in sys.modules)
print(math.pi)
```

Project layout: a directory with `__init__.py` is a package (namespace packages can omit it). `uv` run puts your project on `sys.path`. Do not mutate `sys.path` by hand if a package layout can do the job.

Planned depth (later)

- Finders, loaders, spec

- Relative imports (`from . import util`)
- `importlib.reload` and why it surprises
- Zip imports, editable installs, namespace packages

Further reading

- [Modules](#)
- [importlib](#)

Descriptors and Slots

Descriptors and Slots

`obj.x` is not a field lookup in the C sense. It is a search: instance dict, then type, then **descriptor** protocol (`__get__` / `__set__`). `@property` and bound methods are descriptors.

Mental model

```
obj.x
|
+-- type(obj).__dict__["x"] has __get__? → call it (functions, property)
+-- else obj.__dict__["x"]
+-- else parent types
```

What to use today

```
# celsius_prop.py
class Celsius:
    def __init__(self, c):
        self._c = c

    @property
    def fahrenheit(self):
        return self._c * 9 / 5 + 32

if __name__ == "__main__":
    print(Celsius(0).fahrenheit)
    print(Celsius(20).fahrenheit)
```

Expected:

```
32.0
68.0
```

`__slots__` replaces the instance dict with a fixed layout (less memory, no arbitrary attributes). Use it when you have millions of small objects, not as a style rule.

Planned depth (later)

- Data vs non-data descriptors
- `object.__getattr__`
- How `super()` walks the MRO
- `__slots__` plus inheritance traps

Pair with Objects and Classes once that section is filled.

Further reading

- [Descriptor HOWTO](#)
- Ramalho, *Fluent Python* 2e — descriptors and class metaprogramming

Standard Library

Intro

Standard Library Tour

A map of **when to reach for which module** before you add a dependency. Language → Stdlib essentials is the working set; this part is the index.

Planned chapters

Cluster	Modules
Text and I/O	pathlib, io, json, csv, tomlib
Time	datetime, zoneinfo, time
Process	os, subprocess, signal, argparse
Concurrency	threading, multiprocessing, asyncio, queue
Data	collections, heapq, array, dataclasses
Quality	unittest, doctest, logging, warnings
Net	socket, ssl, http, urllib, email
Internals	sys, gc, dis, inspect, importlib

Official tours: [part 1](#), [part 2](#), then the [library index](#).

Projects

Intro

Projects

Build practice that uses **Language** and the **standard library**. Each project should be small enough to finish and large enough to embarrass you once.

Planned labs

Project	After you have	Ships
Temperature CLI	Getting started, functions	<code>celsius.py</code> with tests
Rename / sort a download folder	<code>pathlib</code> , <code>regex</code>	A script you would keep
Word frequency	<code>dicts</code> , <code>files</code>	Histogram + tests
JSON / CSV rollup	<code>json</code> / <code>csv</code> in the <code>stdlib</code>	A summary you can print

Until more leaves exist: take the **First program** converter, add `f_to_c`, add `pytest`, and put it in a `uv` project. That *is* project 1.